

第3章 Vivado Design Suite 开发平台

本章重点介绍 Xilinx FPGA 开发所需要的软件平台——集成开发环境 Vivado Design Suite（Vivado 设计套件，后简称 Vivado）；以一个实例说明了使用 Vivado 进行 FPGA 硬件开发的流程。

3.1 Vivado Design Suite 平台简介

如 1.2.2 节所述，使用可编程逻辑器件实现电路设计时，必须通过相应的设计开发软件平台来完成。Xilinx（赛灵思）始创于 1984 年，2022 年 AMD 宣布完成了对 Xilinx 的收购。Xilinx 是全球领先的可编程逻辑完整解决方案的供应商，研发、制造并销售应用范围广泛的高级集成电路、软件设计工具以及定义系统级功能的 IP（Intellectual Property）核，长期以来一直推动着 FPGA 技术的发展。

Xilinx 的 FPGA 开发工具不断升级，由早期的 Foundation 发展到 ISE 系列（最后版本是 14.7，2014 年停止升级），2012 年发布了全新的 Xilinx FPGA/CPLD 集成设计环境——Vivado 设计套件（Vivado Design Suite），该平台集成了设计、输入、仿真、逻辑综合、布局布线与实现、时序分析、芯片下载与配置、功率分析等几乎所有设计流程所需工具。

Vivado 发布十余年来，每年均有更新，性能逐步提高，支持更多新款芯片。2016 年发布的 HLx 版本，带有高层次综合器 HLS（High Level Synthesis），可以直接使用 C、C++ 或 System C 等高级语言进行开发；2021 年发布了 Vivado ML（Machine Learning）版本，提升机器学习算法性能。

Vivado 是一款功能强大、易于使用的 FPGA 设计工具，帮助工程师快速开发高性能、高可靠的 FPGA 系统。相比上一代开发工具 ISE，Vivado 在功能整合、算法改进和运行速度上都有了较大的提高。其主要特点概括如下：

- （1）提供了完整的设计流程，包括设计、仿真、综合、实现和验证等功能。
- （2）支持多种 FPGA 芯片系列，包括 Zynq-7000、Kintex-7、Virtex-7、Artix-7 等。
- （3）支持多种设计语言，包括 Verilog、VHDL、SystemVerilog 等 HDL，甚至 C、C++ 高级语言。
- （4）集成了丰富的 IP 库，包括数字信号处理（DSP）、处理器、图像处理、存储器等 IP 核，支持自定义 IP 核，能满足不同应用的需求，可以方便快捷地构建复杂系统，大大减少工作量。
- （5）提供了全面的仿真和调试工具，包括波形查看器、调试器、性能分析器等。

(6) 紧耦合各种工具, 采用共享内存等高效的方式改进综合和实现算法, 提高设计的性能和可靠性; 并且所有工具都采用工具命令语言 Tcl (Tool Command Language) 接口, 可在 Vivado 图形用户界面中获取、输入和执行 Tcl 命令, 提高开发效率。

(7) 支持多种开发板和外设, 可以方便地进行硬件验证和调试。

Vivado 主要功能包括设计输入、分析、综合、仿真、实现和下载, 涵盖了 FPGA 开发的全过程, 具体说明如下:

(1) 设计输入: Vivado 提供了文本编辑器 (Text Editor), 支持 HDL 代码和引脚约束等设计输入; 并提供了 IP 核集成器 (IP Integrator), 支持图形化方式设计并生成 IP 核, 但是不支持原理图的设计输入。

(2) RTL 分析 (RTL Analysis): 对设计好的工程进行 RTL 分析、语法检查等, 生成 RTL 结构图 (与 FPGA 底层部件无关的逻辑符号组成的原理图), 供用户查看, 进而可以分析并修正逻辑上的错误。

(3) 综合 (Synthesis): 将 RTL 综合出来的原理图网表映射到 FPGA 中的底层部件 (查找表 LUT), 并转换为综合网表 (NetList), 包含工程设计所对应的资源模块 (组合逻辑器件、寄存器、原语、内部的硬核模块等) 及其属性和约束信息描述。

(4) 仿真 (Simulation): 使用专门的仿真工具进行模拟仿真, 验证设计的功能是否正确, 包括行为仿真、功能仿真和时序仿真等。Vivado 有自己的 Vivado Simulator, 也支持联用众多第三方产品, 包括 Mentor Graphics、Cadence、Synopsys 和 Aldec 等公司仿真器和模拟器, 设计者可以自行选择。

(5) 实现 (Implementation): 此功能针对具体指定的器件类型及相关物理与性能约束, 由 FPGA 实现工具将综合后的电路网表进行优化、布局、布线, 并生成包含物理数据的网表文件。该过程包括优化设计、功耗优化、布局设计、布局优化、布线设计、布线优化等。

(6) 比特流生成 (Generate Bitstream): 将实现产生的网表文件转化为比特流文件 (配置文件)。

(7) 编程下载 (Program Device): 将比特流文件下载到 FPGA 芯片中。

Vivado Design Suite 作为高效的 EDA 设计工具集合, 具有界面友好、操作简单的特点, 加之 Xilinx 的 FPGA 芯片占有很大的市场, 使其成为非常通用的 FPGA 工具软件。各个版本的 Vivado 软件 (包括免费版本) 均可从 Xilinx 官网 (<https://china.xilinx.com/support/download.html>) 上下载, 请读者自行选择下载并安装。各版本软件从 30GB~110GB 不等, 安装中可以取消勾选部分选项, 选择部分安装。注意: **本教材使用的 FPGA 实验板卡的芯片系列为 Artix-7**, 请务必选择安装支持该系列芯片。

3.2 Vivado 项目开发流程

3.2.1 开发流程概述

图 3.1 概括性地展示了使用 Vivado 进行 FPGA 项目开发的流程，主要包括设计输入、综合（Synthesis）、实现（Implementation）、生成比特流（Generate Bitstream）、下载（Program Device）等步骤。在将设计下载到器件之前，一般需要进行仿真（Simulation）来验证项目设计是否正确；仿真验证分为行为仿真、功能仿真和时序仿真，可以选择在某个阶段进行，也可以不仿真。

下面我们以一个完整的实例，介绍如何使用 Vivado Design Suite，完成一个项目开发。本节图片示例使用 Vivado 2018.3 版本，代码为例【2.1】中的 First_M 模块。

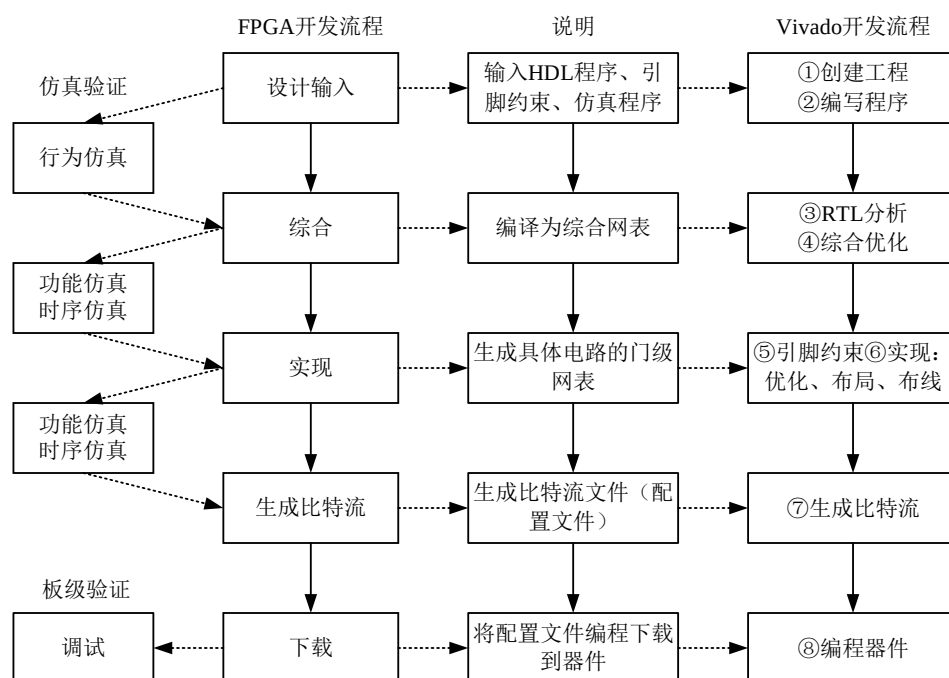


图3.1 使用 Vivado 开发工程项目的流程

3.2.2 创建工程

(1) 新建工程。启动 Vivado Design Suite 软件，每次打开时都会在起始界面右侧显示最近打开过的工程列表，可以直接单击某个工程再次打开它。这里选择创建一个新工程，如图 3.2 所示，单击“Create Project”；弹出“New Project”窗口，单击“Next”按钮，进行下一步。

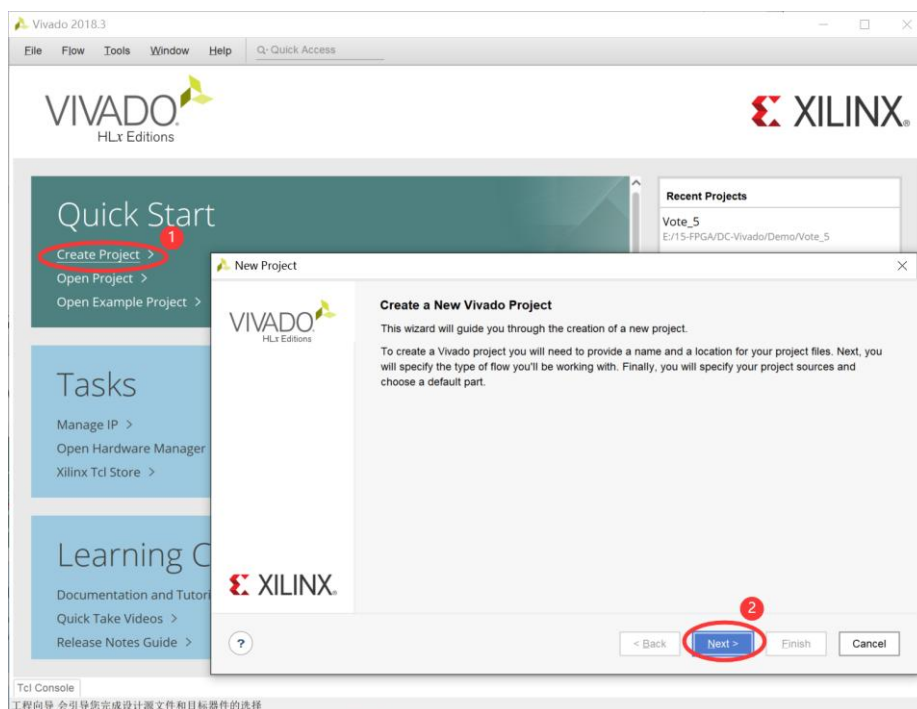


图3.2 创建新工程

(2) **指定工程名称和路径。**在弹出的对话框中，“Project name”中输入工程名称，这里假设为“First_Project”；然后选择新建工程所在的路径（可以点击右边的“...”按钮浏览选择）；最后点击“Next”进行下一步，如图 3.3 所示。注意：文件目录的路径上不能有中文字符。

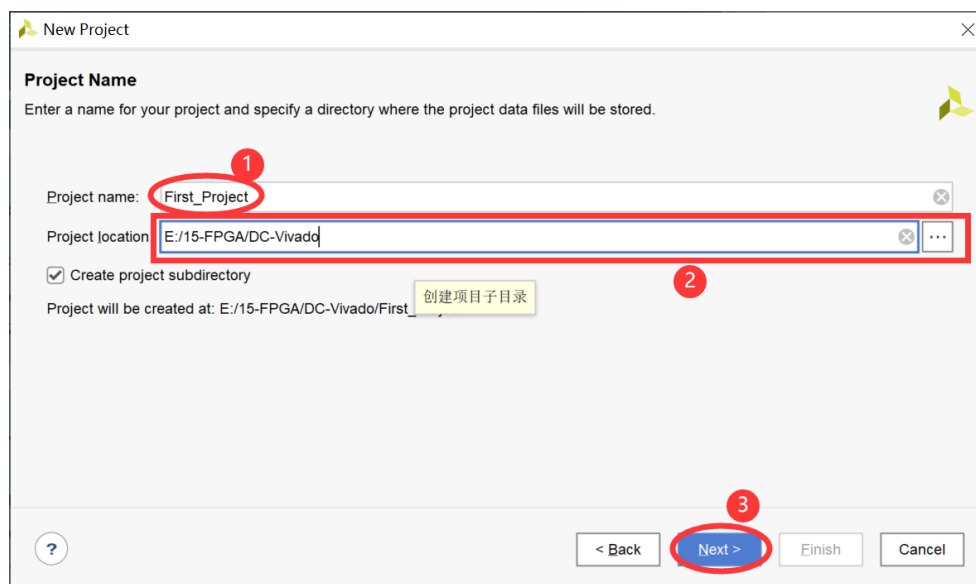


图3.3 指定新工程名称和路径

(3) **选择工程类型。**弹出“Project Type”对话框，如图 3.4 所示。选中“RTL Project”类型，点击“Next”进行下一步。

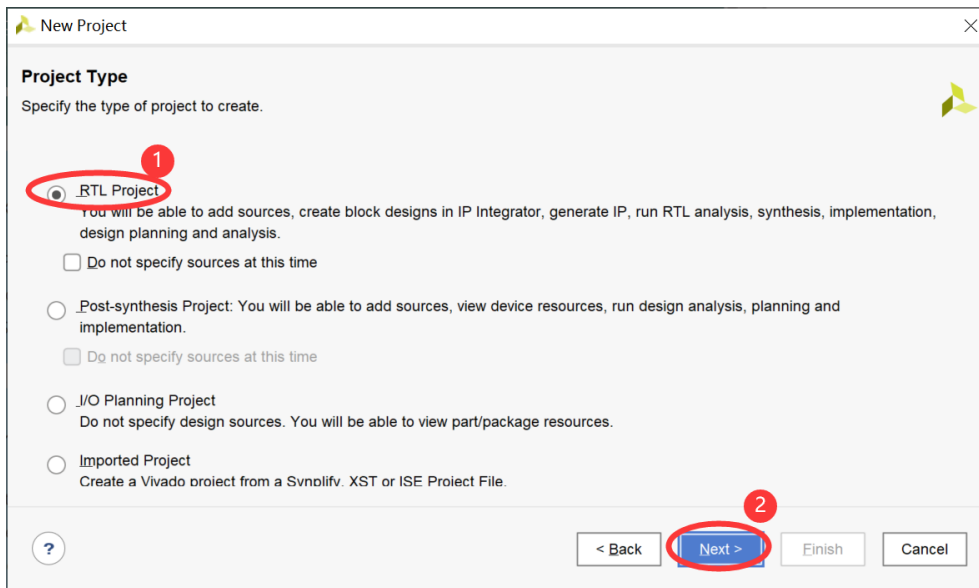


图3.4 选择工程类型

(4) 添加或者新建源程序。弹出“Add Sources”对话框，如图 3.5 所示，下方默认选择的目标语言是 Verilog，仿真语言是“Mixed”。在此可以点击“Add Files”或者“Add Directories”添加已有的源程序或者目录到该工程项目中；本次示例选择点击“Create File”按钮以新建一个源程序，单击后弹出“Create Source File”对话框如图 3.6 所示。本步骤也可以在图 3.5 所示的对话框上直接点击“Next”进行下一步，这样就建立了一个没有源文件的空工程。

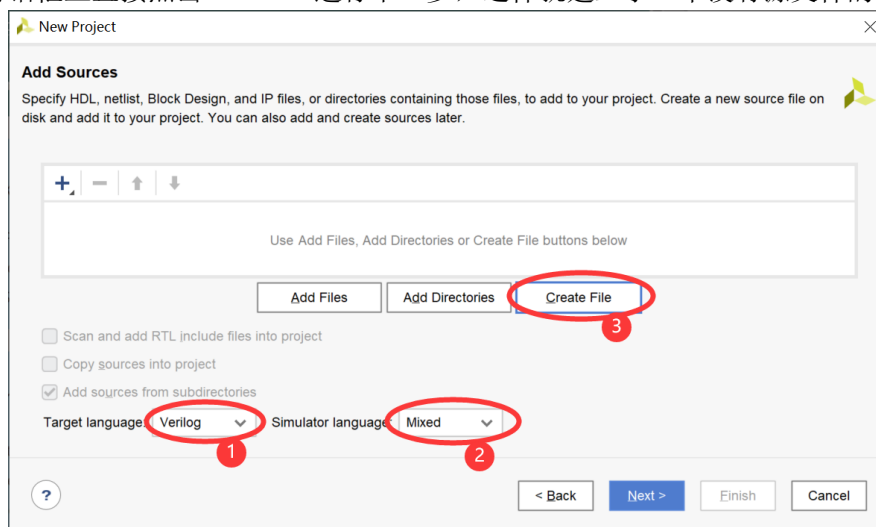


图3.5 Add Sources 对话框

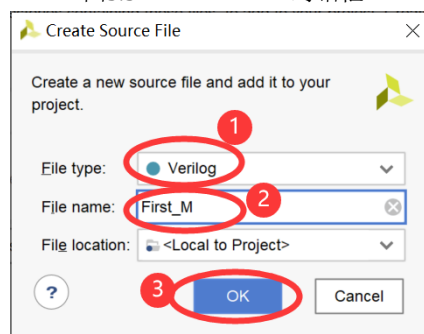


图3.6 新建 Verilog HDL 源程序

在图 3.6 的对话框上，默认的文件类型是 Verilog，这意味着生成的文件是*.v 文件；然后输入文件名，假设输入“First_M”，然后点击“OK”按钮，对话框关闭，返回到图 3.5 所示对话框，但是新工程中多了“First_M.v”文件，如图 3.7 所示。点击“Next”进行下一步。

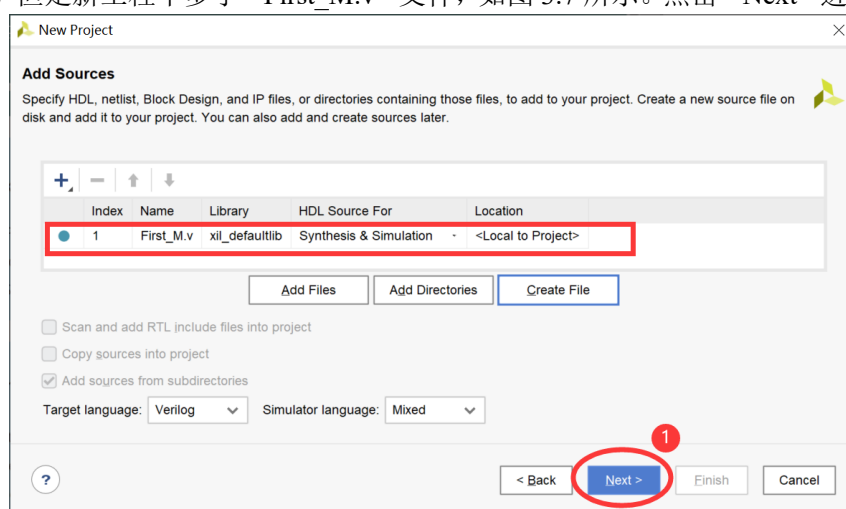


图3.7 新建 First_M.v 文件

(5) 新建或添加约束文件。弹出的对话框如图 3.8，可以选择点击“Add Files”添加已有的约束文件到工程项目中，或者点击“Create File”按钮新建一个约束文件到工程项目中。约束文件的作用是：将工程项目顶层模块的输入输出端口，一一配置（约束）到 FPGA 芯片的指定引脚上。这里我们点击“Next”进行下一步，即：后续再添加约束文件。

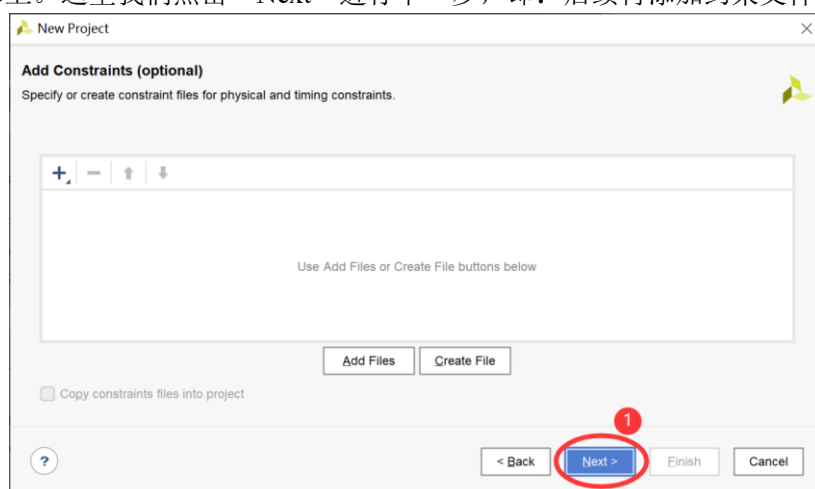


图3.8 添加或者新建约束文件

(6) 选择 Xilinx FPGA 器件或板卡型号。对话框如图 3.9 所示，上面有两个标签页，Parts 标签页用于选择 Xilinx FPGA 器件型号，Boards 标签页则选择 Xilinx 开发板的型号。这里选择器件型号，先选择系列（Family），本教材介绍的几种实验板卡均是 Artix-7 系列，因此在下拉式菜单中选择“Artix-7”；然后再选中下面该系列芯片列表中对应的芯片型号：如果使用的是 HCS-A02 或者 NEXYS A7 实验板卡，应选择 xc7a100tcsg324-2L 芯片。这一步一定要选对芯片型号，否则后续引脚约束和下载不能正确完成。最后点击“Next”进行下一步。

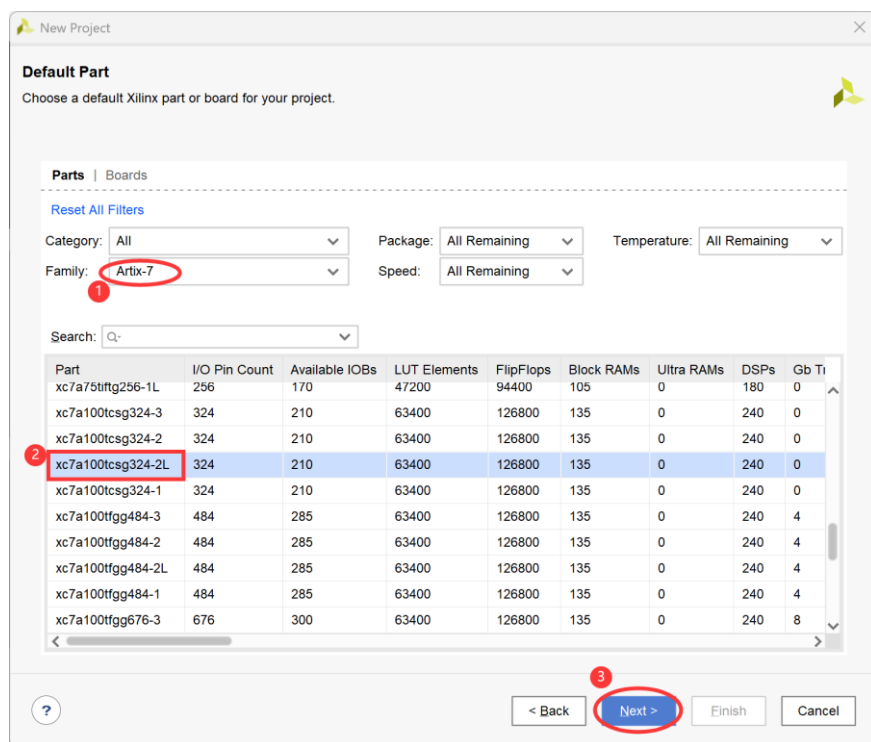


图3.9 选择芯片型号

(7) **完成新建工程。**弹出的对话框概览了新建工程的信息，如图 3.10 所示，直接单击“Finish”按钮，就完成了这个新工程的创建工作。这时，在前述指定的目录下，就自动建立了一些子目录和“First_Project.xpr”工程文件。

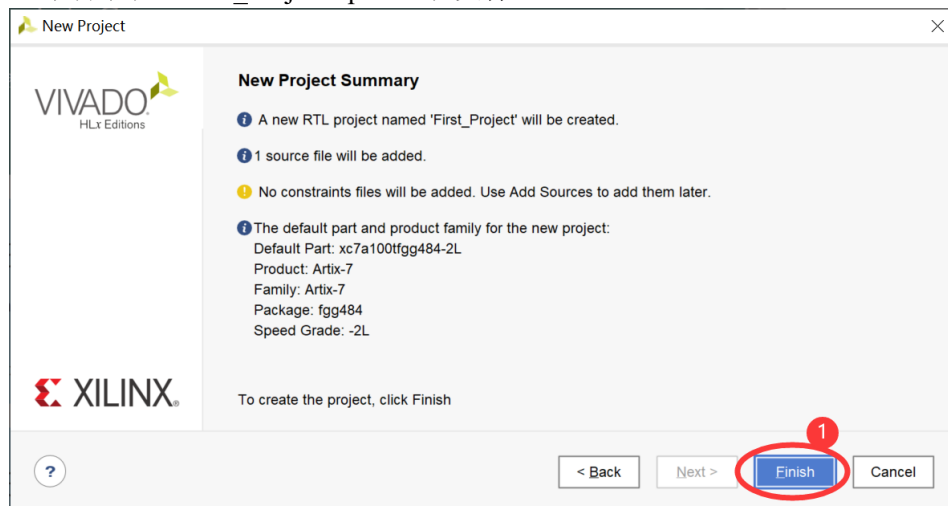


图3.10 新建工程总览

(8) **创建工程的 HDL 模块。**上一步单击“Finish”按钮后，弹出图 3.11 所示的对话框，用于定义工程的 HDL 模块，其中 Module name 栏用于输入模块名，默认模块名是之前新建的*.v 文件的文件名，即：First_M；在下面可以定义模块的端口名（Port Name）、选择端口输入输出方向（Direction，input、output 或 inout）、勾选位宽是否多位（即总线 Bus），如果是总线型，则可以输入最高序号（MSB）和最低序号（LSB）。这里也可以不操作，直接点击“OK”，后续编写源程序时再定义模块端口。

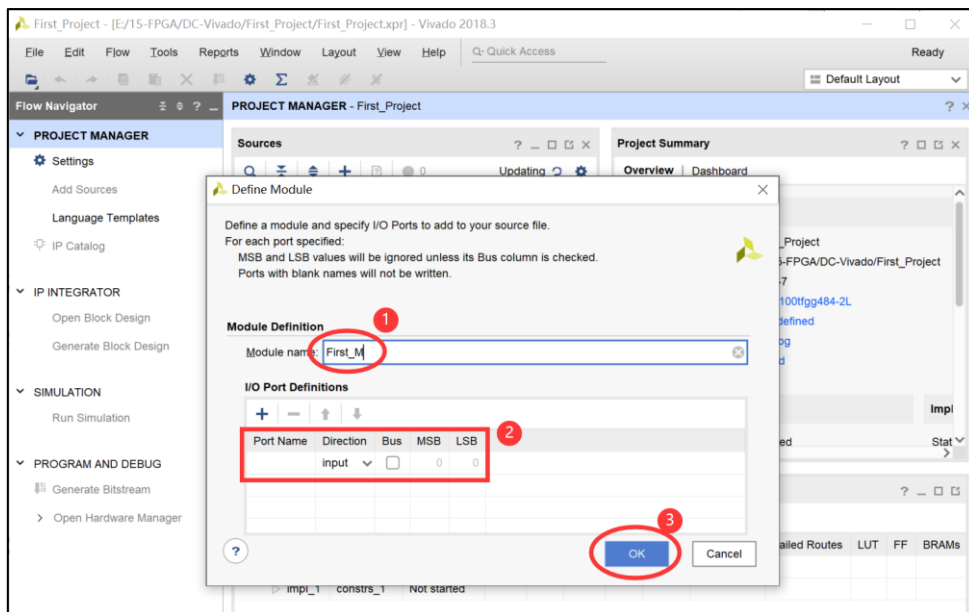


图3.11 创建 Verilog HDL 模块

(9) 打开工程管理主界面。上一步单击“OK”按钮后就弹出了工程管理主界面，如图 3.12 所示。打开已有工程的方式有两种：一是在工程所在目录下双击工程文件*.xpr，二是打开 Vivado 软件，在图 3.2 所示界面上，单击“Open Project”，到工程所在目录下选择相应的工程文件。

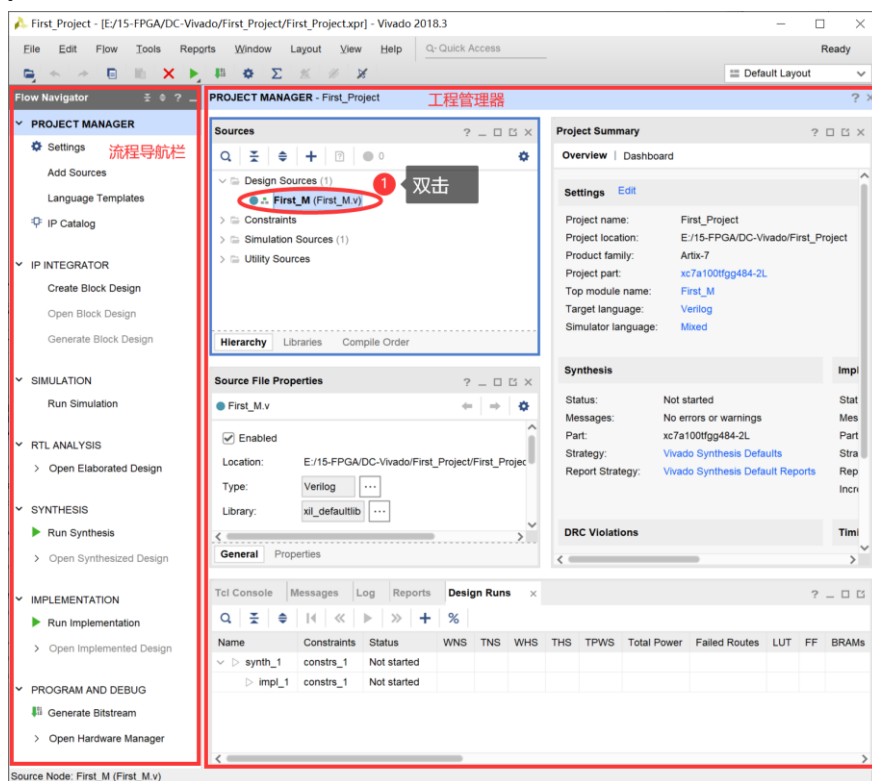


图3.12 已创建的新工程

图 3.12 所示的工程管理主界面上，左边是开发流程导航栏（Flow Navigator），按照从上到下顺序基本可以完成整个项目的开发（如图 3.1 所示）。右边是工程项目管理器，包含很

多子窗口。在“Sources”子窗口下，可以看到，我们之前创建的模块 First_M，它位于源代码文件 First_M.v 中，双击模块 First_M，将打开 First_M.v 文件，如图 3.13 所示，进入了模块代码的文本编辑器。

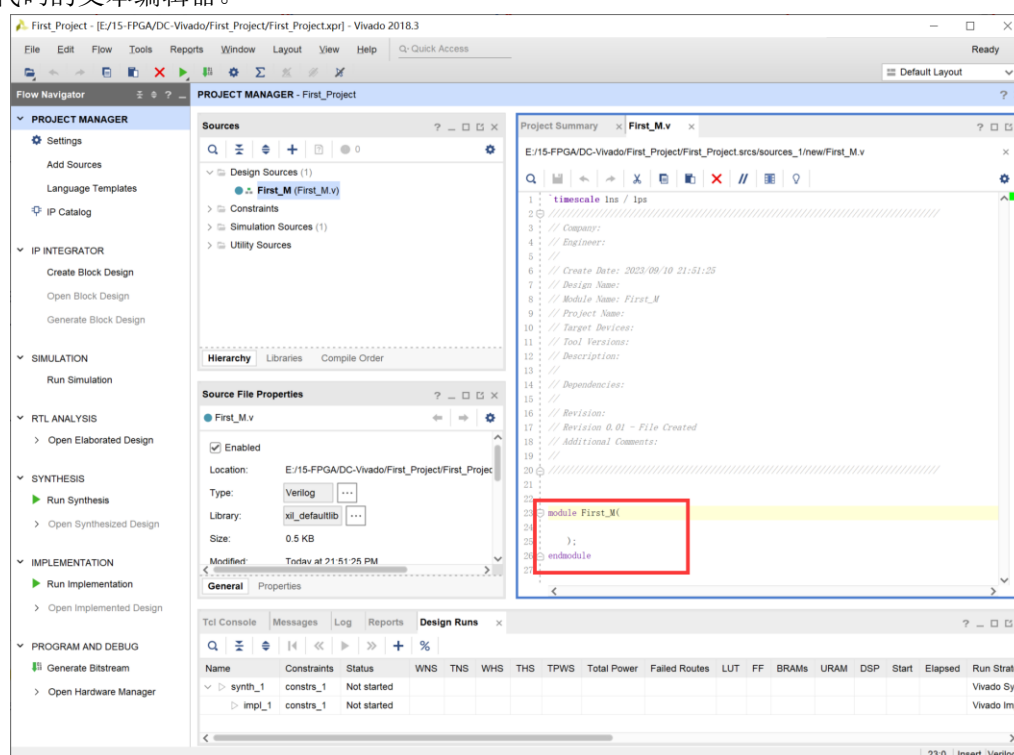


图3.13 打开模块的文本编辑器

3.2.3 编写代码

(1) **编写程序代码。**在图 3.13 的 First_M.v 文件的文本编辑器中，可见前述新建模块 First_M 是空的，按照 Verilog HDL 语法编写并输入【例 2.1】中 First_M 模块的程序，如下所示。

```
module First_M (
    input A,
    input B,
    input C,
    input D,
    input E,
    output F
);
    assign F = ~((A & B & C) | (D & E));
endmodule
```

(2) **修正错误，保存文件。**在代码输入过程中，有明显的语法错误时，会在相应代码下出现红色的波浪下画线，鼠标移至红色波浪线上，会弹出对应的错误提示，如图 3.14 所示。修正后，单击“保存”按钮或者按下 ctrl+S 键，将保存当前文件。

(3) **查看错误及其他信息。**保存文件后，Vivado 会自动扫描源程序，进行语法分析与检查。单击下方的控制台窗口的“Messages”页，会显示 Vivado 执行的命令（位于上方）以

及执行的结果提示（位于下方），如图 3.15 所示，其中结果中就包括了出错信息，具体含出错类型（Warning/Critical warning/Error）、错误具体信息、出错文件名及错误代码位于文件的第几行。单击蓝色的文件名和行序号（图 3.15 的“First_M.v:23”），可以直接将光标定位到相应的出错文件和相应行。

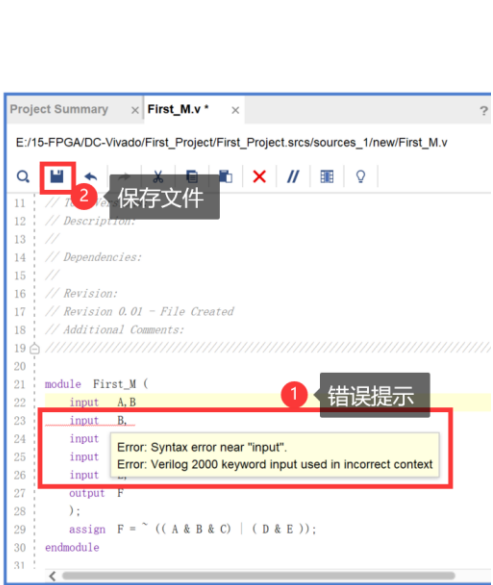


图3.14 代码错误提示及保存

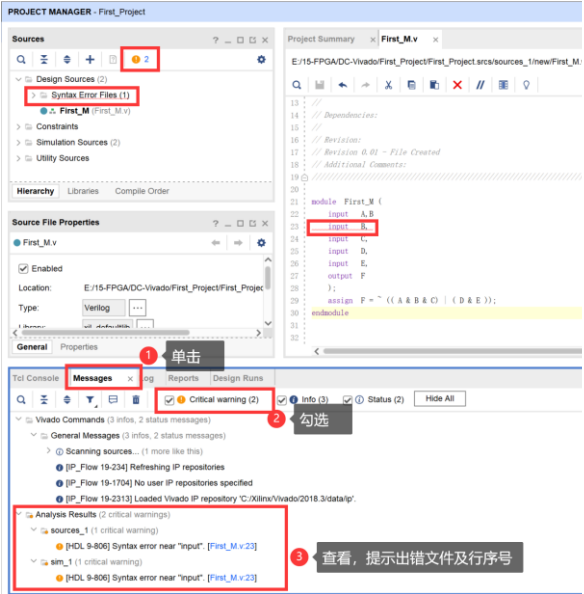


图3.15 查看错误信息及其他信息

3.2.4 查看电路图

当输入的源程序完全正确后，可以通过依次单击左侧导航栏的“RTL ANALYSIS→Open Elaborated Design→Schematic”来查看 RTL（Register Transfer Level，寄存器传输级）电路图，如图 3.16 所示；在弹出的对话框中，单击“OK”后，Vivado 将绘制工程的顶层模块 RTL 逻辑图。

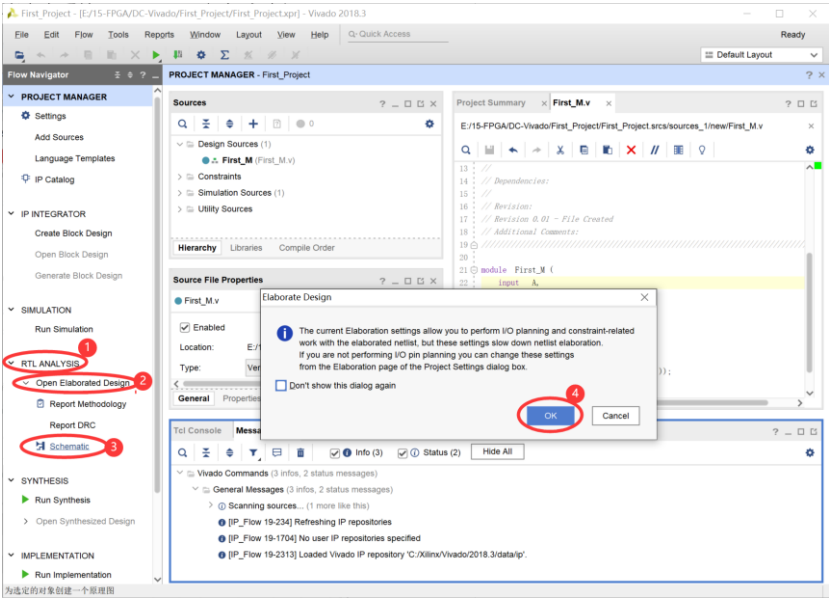
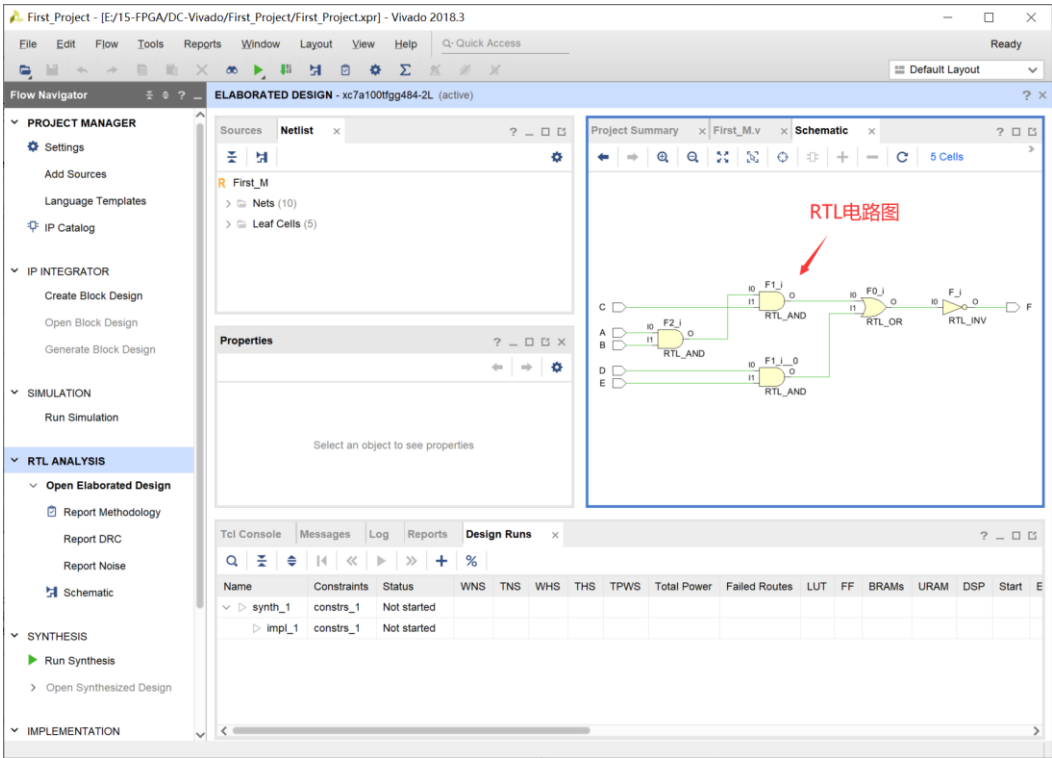
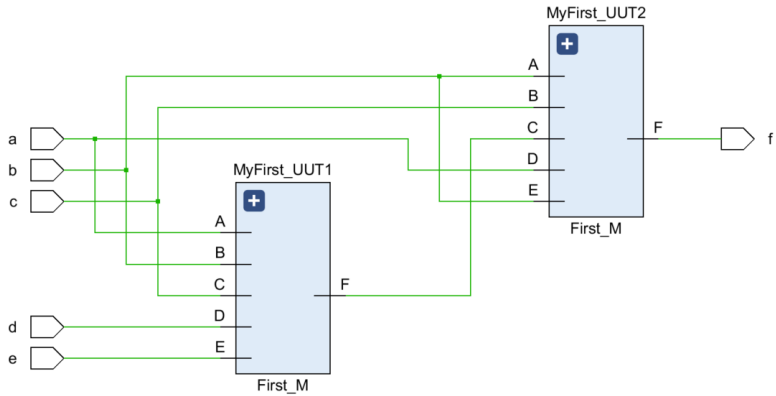


图3.16 查看 RTL 电路图

First_M 模块的 RTL 电路图如图 3.17 (a) 所示, 包含了 3 个与门、1 个或门和 1 个非门, 显然符合程序的逻辑功能定义: $F = \overline{ABC} + \overline{DE}$ 。点击“ELABORATED DESIGN”窗口右侧的“×”可关闭电路图绘制。RTL 电路图与 FPGA 器件无关, 仅仅是电路原理图, 供设计者检查设计及模块间的连接是否正确。



(a) First_M 模块的 RTL 电路图



(b) 【思考与实践 2.3】First_M_Inst2 模块的 RTL 电路图

图3.17 RTL 电路图

包含多个模块的工程中, 有一个为**顶层模块** (Top Module): 调用别的子模块的模块会自动变为顶层模块, 也可以指定某模块为顶层模块 (选中该模块, 右键点击, 弹出菜单中单击“Set as Top”)。RTL 原理图会显示顶层模块所包含的所有模块间的连接视图。例如【思考与实践 2.3】中, 顶层模块 First_M_Inst2 包含了两个 First_M 子模块, 其 RTL 视图如图 3.17 (b) 所示, 清楚展示了两个模块间的连接, 以及输入输出端口; 单击子模块上的“+”号, 可展开该模块内的逻辑电路图。

在对工程项目进行综合和实现后，还可以通过在导航栏依次单击“SYNTHESIS→Open Synthesized Design→Schematic”，或者“IMPLEMENTATION→Open implemented Design→Schematic”来查看综合后或者实现后的电路图，与器件有关系，故电路会有所不同。

3.2.5 行为仿真

下面对工程的 First_M 模块进行行为仿真，验证其功能是否正确。

(1) 新建仿真源程序。在左侧的工程导航栏，如图 3.18 所示，单击“Add Sources”，并在弹出的对话框中选择“Add or create simulation sources”，即添加或新建仿真源程序，最后单击“Next”。

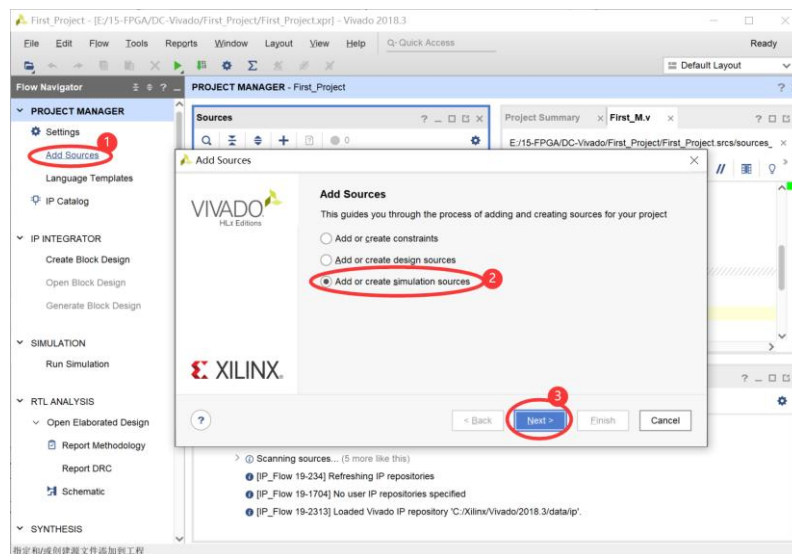


图3.18 新建仿真源程序

(2) 输入仿真文件名。弹出对话框如错误!未找到引用源。所示，单击“Create File”后，则新弹出一个对话框，在其中输入仿真文件名称，假设为“First_M_Test”，单击“OK”后对话框关闭。这时在 sim_1 仿真系统下已经新添加了“First_M_Test.v”文件，如图 3.20 所示。最后单击“Finish”按钮。

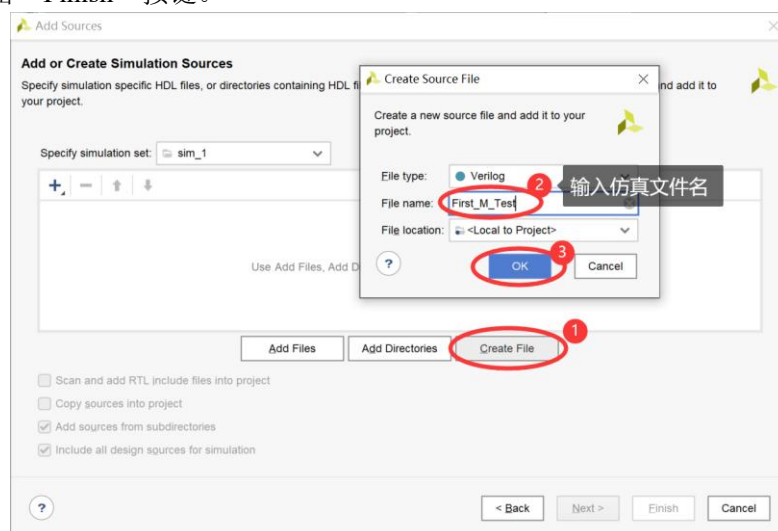


图3.19 新建并输入仿真文件名

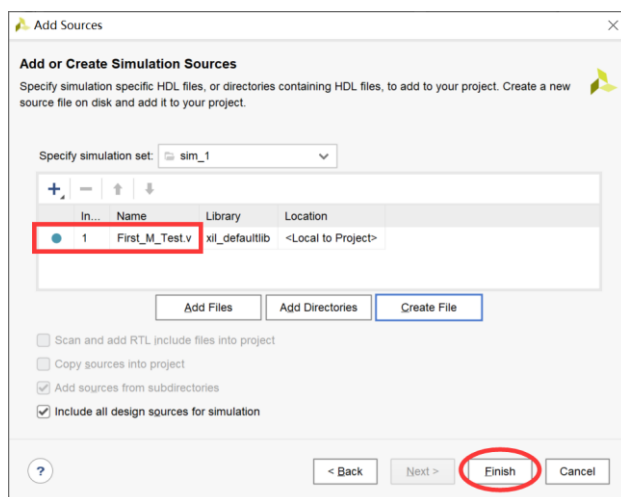


图3.20 结束仿真文件创建

(3) **定义仿真模块名称及端口。**继续弹出对话框，如图 3.21 所示，可以输入仿真源程序中的模块名称，默认与仿真源程序名称一致，即“First_M_Test”；也可以继续定义该仿真模块的端口名、类型等属性，图 3.21 中未定义端口，可以后续在程序编辑器中直接输入。最后单击“OK”，弹出确认对话框，单击“Yes”结束仿真源程序及模块的创建。



图3.21 定义仿真模块名称和端口

(4) **输入仿真代码。**在“Sources”子窗口下的“Simulation Sources”树形浏览器中，双击刚才创建的“First_M_Test”模块，如图 3.22 所示，在右侧的程序编辑器中可以输入以下仿真代码。

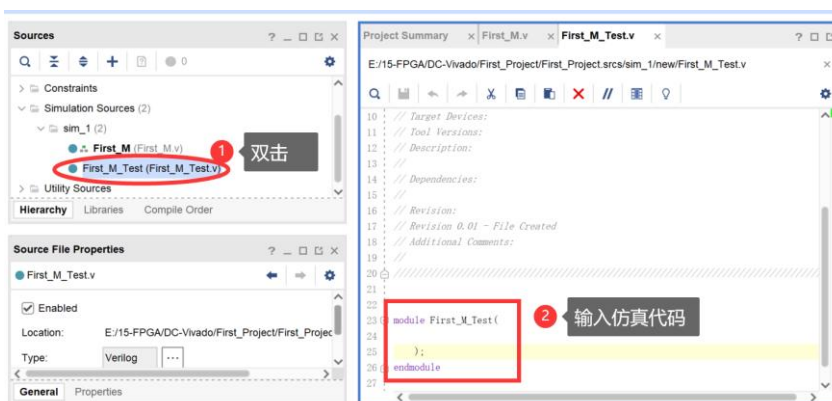


图3.22 输入仿真源代码

```

module First_M_Test;    //仿真的顶层模块，没有输入输出端口
    reg      A,B,C,D,E;    //在 initial 中被赋值，故必须定义为 reg 类型
    wire      F;           //要连接到 First_M 的输出端口 F，故必须定义为 wire 类型
    First_M    The_UUT(A,B,C,D,E,F);    //定义了模块 First_M 的一个实例 The_UUT
    initial    //测试样例
    begin
        A=0; B=0; C=0; D=0; E=0;        //第 1 组测试：初始化为全零
        #100; A=0; B=0; C=0; D=0; E=1;    //第 2 组测试
        #100; A=0; B=0; C=0; D=1; E=1;    //第 3 组测试
        #100; A=1; B=0; C=0; D=0; E=0;    //第 4 组测试
        #100; A=1; B=1; C=0; D=0; E=0;    //第 5 组测试
        #100; A=1; B=1; C=1; D=0; E=0;    //第 6 组测试
    end
endmodule

```

分析这段仿真代码，理解仿真的原理：首先，调用被仿真的模块 First_M 生成一个模块实例 The_UUT，程序中事先定义了变量 A~F 作为 The_UUT 的端口，按照顺序连接到 First_M 模块的端口 A~F。其次，在 initial 模块中，先对输入变量 A~E 进行全零初始化测试，然后每隔 100ns 依次赋值不同数据，故共计测试了 6 组数值。

(5) 行为仿真。首先在“Sources”子窗口中选中仿真顶层模块 First_M_Test，然后工程导航栏中，依次单击“SIMULATION→Run Simulation→Run Behavioral Simulation”，如图 3.23 所示，则就会运行行为仿真代码，稍等片刻，即可产生波形图，如图 3.24 所示。

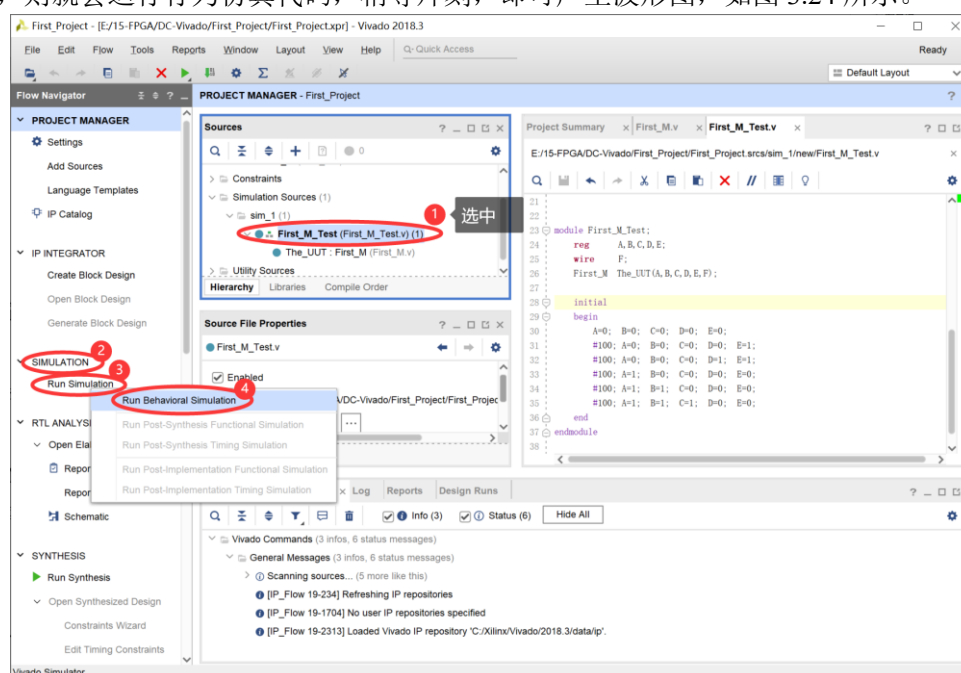


图3.23 运行行为仿真

初始仿真波形为直线，在图 3.24 上，单击“Zoom Fit”图标 ，即可放大波形；单击波形窗口的最大化图标 ，可以隐藏“Scope”、“Objects”、“Tcl Console”等子窗口，让仿真波形窗口最大化；默认波形显示 1000ns，可以单击“Run for 10 us”图标 ，产生更多的波形，其中再运行产生波形的时间“10”和“us”是由该图标右侧的输入框和下拉框指定的。

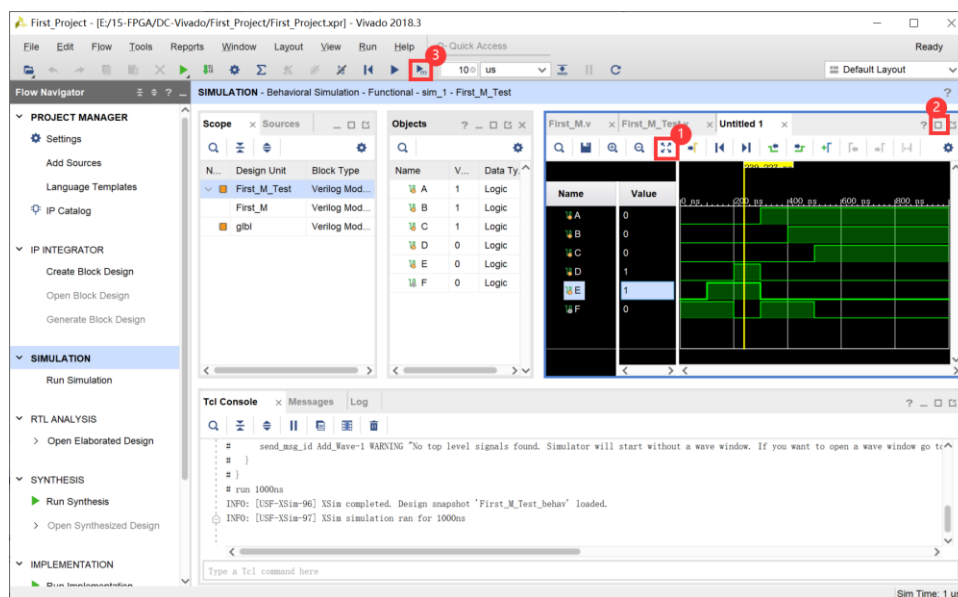


图3.24 仿真波形图

可以拖动图 3.24 上的黄线，在左边查看各变量的值，也可以直接观察波形，据此分析在当前输入下，输出是否正确。例如：在 0ns 时，ABCDE=00000，F=1；100ns 时，ABCDE=00001，F=1；200ns 时，ABCDE=00011，F=0；300ns 时，ABCDE=10000，F=1；400ns 时，ABCDE=11000，F=1；500ns 时，ABCDE=11100，F=0；显然，均符合 $F = \overline{ABC} + \overline{DE}$ 的逻辑关系。

(6) 组合逻辑电路的全仿真。上述仿真代码只抽样检查了 6 组测试值，属于部分仿真，不能完全验证所有的输入组合都正确。如果要按照真值表进行全部的测试，需要写 32 条赋值语句，太冗繁。下面的代码很方便地实现全仿真：将 5 个输入变量用“{}”连接成一个整体，在进行初始化置全零后，使用另一段 always 语句对它们执行每隔 100ns 就加 1 的操作，这样经过 3200ns 后，所有的输入组合全部遍历测试一遍，波形如图 3.25 所示。按照真值表分析波形，可以验证 First_M 模块的逻辑功能正确。

```
module First_M_Test;
    reg      A,B,C,D,E;
    wire      F;
    First_M The_UUT(A,B,C,D,E,F);
    initial {A,B,C,D,E} = 5'b0; //连接 A~E 五个变量，一同赋初值=00000
    always    //没有敏感事件列表@，始终执行
    begin
        #100;
        {A,B,C,D,E} = {A,B,C,D,E} + 1'b1;
    end
endmodule
```

可以点击“Zoom Out”、“Zoom In”按钮放大、缩小波形图的间隔时间。在波形图的左侧，可以单击“Scope”、“Sources”、“Objects”等标签，显示相应的子窗口。

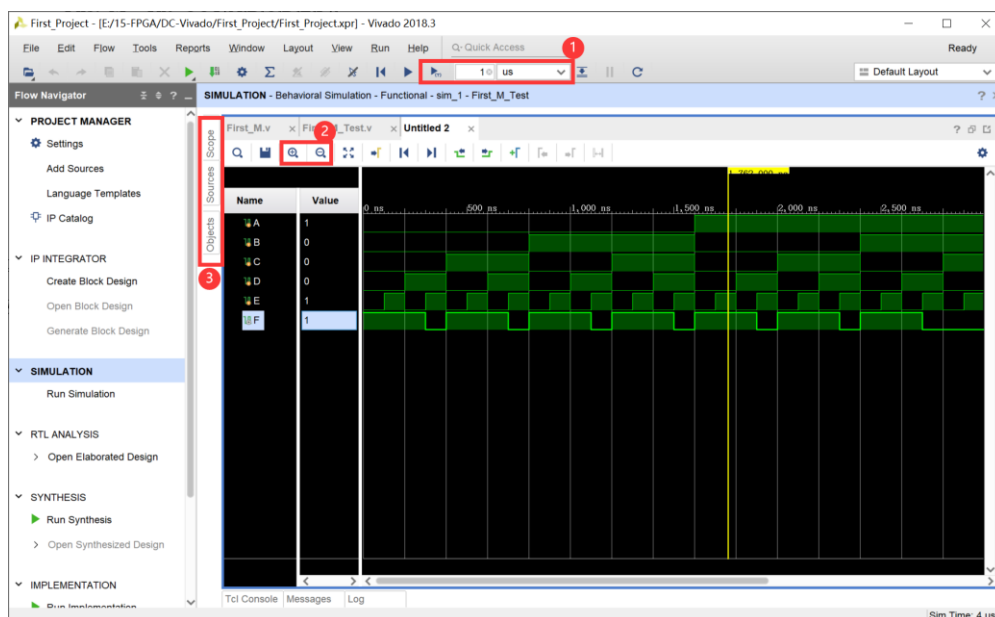


图3.25 First_M 的全仿真波形图

(7) 同步时序逻辑电路的仿真。对于同步时序逻辑电路的仿真，因为存在时钟信号，故仿真测试代码有较大不同，尤其是边沿触发的时序电路。在测试某组输入组合时，要让时钟信号发生有效的变化，才能测试电路行为是否正确。以【例 2.16】中的带异步清零端的边沿 D 触发器模块 UP_DFF_RD_asyn 为例，我们编写其测试仿真代码如下：

```
module UP_DFF_RD_asyn_Test;
    reg    CP,RD,_D;
    wire    Q,Q_;
    UP_DFF_RD_asyn  The_DFF_UUT(CP,RD,_D,Q,Q_);
    initial
    begin
        CP = 0;    RD_ = 0;    D = 0; //①初始化，异步清零信号有效：Q = 0
        #50; D = 1;    //②测试 D=1 时，Q 是否有变化：Q=0，因为处于异步清零状态下
        #50; CP = 1;    //③测试 D=1 时，CP 上升沿来临时 Q 是否有变化：Q=0，因为处于异步清
        零状态下
        #50; RD_ = 1;    //④测试不清零状态：Q=0 保持不变，因为 CP 无有效跳变
        #50; CP = 0;    //⑤测试 CP=0、下跳沿来临时 Q 是否有变化：Q=0 无变化
        #50; CP = 1;    //⑥测试 CP 上升沿来临时 Q 是否有变化：Q=D=1
        #50; D = 0;    //⑦D=0：Q=1 无变化，因为 CP 无有效跳变
        #50; CP = 0;    //CP=0：Q=1 无变化
        #50; CP = 1;    //⑧CP 上升沿：Q=0
        #50; CP = 0; D = 1;
        #50; CP = 1;    //⑨CP 上升沿：Q=D=1
        #50; RD_ = 0;    //⑩异步清零：Q=0
    end
endmodule
```

分析上述仿真代码：首先初始化时钟信号 CP=0、激励输入 D=0、异步清零 RD_=0。如果时序逻辑电路有异步清零信号，因为其，一般要先将其置为有效电平，保证时序电路从稳定的初态开始进行状态转移；仿真验证时应如此设置，板级验证时也应如此操作。因此，

上述 D 触发器从仿真零时刻第①行开始，就被清零了。后续的②~③行，测试异步清零信号 RD_ 是否比 CP 的优先级高，即：处于异步清零状态下，CP 与 D 的变化不会引起 Q 的变化。第④行的 RD_ 被置为无效的 1，电路处于正常的工作状态，此后第⑤~⑨行，就测试 Q 如何随 CP 和 D 的变化而变化的行为。这里一定注意，一般情况下，先改变激励输入 D 的值，再给出 CP 的上升沿，即：**在 CP 有效边沿来临时，激励输入 D 要保持稳定（不变）。**

上述仿真代码中，CP 手动赋值 0/1 来产生上升沿，实际上可以通过 always 语句将其设置为周期性变化的值，如下所示。

```

module UP_DFF_RD_asyn_Test;
    reg    CP,RD_,D;
    wire    Q,Q_;
    UP_DFF_RD_asyn  The_DFF_UUT(CP,RD_,D, Q,Q_);图 3.26
    initial
    begin
        CP = 0; RD_ = 0;    D = 0; //初始化，异步清零信号有效:Q=0
        #25; D = 1; //D=1，观察后续 CP 上升沿来临时，Q 是否变化：Q=0 保持不变（异步清
        零状态）
        #100; RD_ = 1; //变为不清零状态（正常工作），观察后续 CP 上升沿来临时，Q 是否变
        化：Q=1;
        #100; D = 0;        //D=0，观察后续 CP 上升沿来临时，Q 是否变化：Q=0;
        #100; D = 1;        //D=1，观察后续 CP 上升沿来临时，Q 是否变化：Q=1;
        #100; RD_ = 0;      //异步清零：Q=0
    end
    always    begin
        #50; CP = ~CP;      //CP 为周期性信号，时钟周期为 100ns
    end
endmodule

```

注意：①其中 initial 和 always 同时从仿真零时刻开始执行，always 始终循环不断地执行，而 initial 则只执行一次。故 CP 从初始置为 0 后，每过 50ns 就执行一次取反操作，产生了周期为 100ns 的时钟信号。②激励输入 D 的变化时刻不要和 CP 的有效边沿重合，因此第一次延时选择 25ns 而不是 50ns，之后每隔 100ns 改变 D 或者 RD_。上述仿真代码产生的仿真波形如图 3.26 所示。

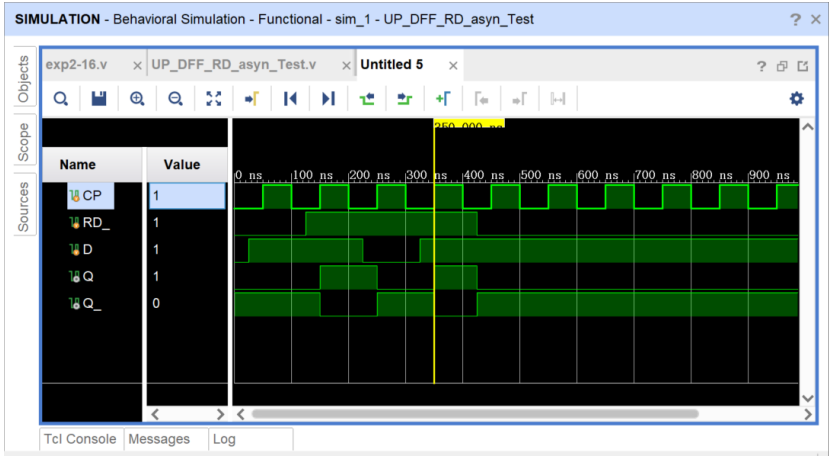


图3.26 【例 2.16】UP_DFF_RD_asyn 模块的仿真波形

分析图 3.26 的波形可发现, 0ns 时刻 RD_赋值 0 之后, D 触发器的输出 Q 就清为零了; 即使在 25ns 时刻 D 变为 1, Q 没有改变; 50ns 时刻 CP 上升沿来临, Q 也没有改变。在 125ns 时刻, RD_赋值为 1, 此时 Q 仍保持为零, 直到 150ns, CP 上升沿到达后, 将 D 的值写入 Q, Q 变为 1。之后, 在 250ns 和 350ns 时刻, 各有一个 CP 的上升沿, Q 都随着 D 的改变而改变; 在 425ns 时刻, RD_再次变为 0, 从而使得触发器 Q 清零。因此, 通过以上分析推导, 可以得到结论: 模块 UP_DFF_RD_async 定义的带异步清零端的、正边沿触发的 D 触发器逻辑功能正确。

要想成功完成一个工程项目, 除了前期的需求分析、方案设计与编码实现, 还必须对工程设计进行逻辑严密的全面测试, 然后收集实验数据和结果, 并能对数据进行耐心、仔细、严谨的分析, 最终推导出实验结论: 功能是否完备、逻辑是否正确、运转效率如何等等。因此, 设计实验方案、收集实验数据、分析实验结果、得出实验结论是**计算机工程师必须具备的基本能力**。

【思考与实践 3.1】:在 Vivado 中完成上述仿真实验, 观察你的波形是否与图 3.26 一致?

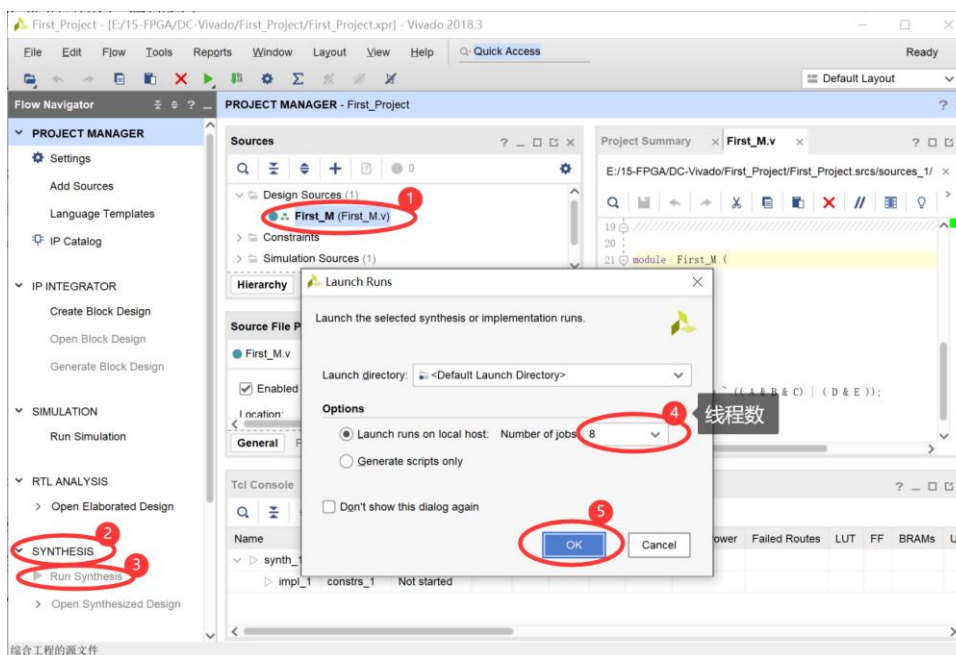
【思考与实践 3.2】:尝试修改上述仿真实验: 在 CP 上升沿来临的同一时刻, 修改 D 的值。观察仿真波形, Q 是等于 D 变化前的值, 还是等于 D 变化后的值? 你觉得理论上 Q 应该取哪个值? 如果结论不一致, 你觉得应当如何解释?

3.2.6 逻辑综合

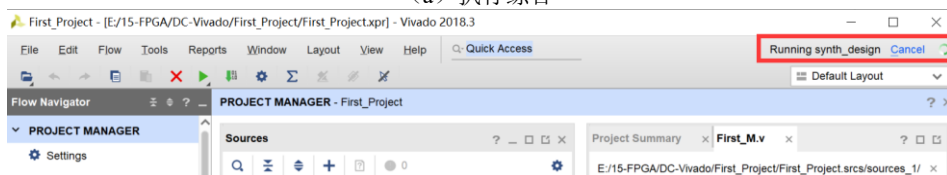
完成了源程序输入后就可以进行综合 (Synthesis), 即: 将 HDL 语言、IP 核的原理图等设计输入翻译成由与、或、非门和 RAM、触发器等基本逻辑单元的逻辑连接 (网表), 并根据目标和要求 (约束条件) 优化所生成的逻辑连接。

选中工程的顶层模块, 单击左侧的导航栏的 “SYNTHESIS→Run Synthesis”, 弹出一个对话框, 如图 3.27 (a) 所示, 输入任务数 (Number of jobs), 其实就是开启的线程数, 可以根据计算机的 CPU 核心数 (Cores) 来设置, 然后单击 “OK”, 则 Vivado 开始运行 synth_design 任务, 图 3.27 (b) 右上角可以看到 Vivado 执行任务的状态 “Running synth_design”。

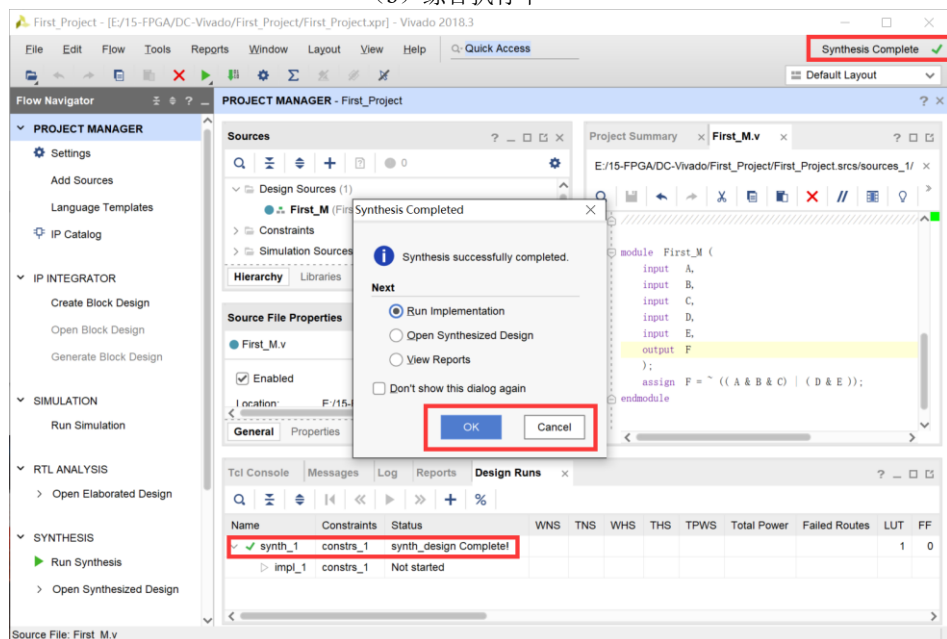
综合任务执行完毕后, 窗口右上角的任务状态会显示 “Synthesis Complete”, 如图 3.27 (c) 所示, 同时会弹出对话框, 询问是否进行下一步的操作 “实现”。可以单击 “OK” 继续 “实现”; 也可以单击 “Cancel” 选择不继续, 下次可通过导航栏选择 “实现” 操作。



(a) 执行综合



(b) 综合执行中



(c) 综合完成

图3.27 执行综合操作

3.2.7 实现

完成综合操作之后，可以执行实现（Implementation）操作。所谓实现，是指将综合输出的逻辑网表翻译成所选器件的底层模块和硬件原语，将设计映射到器件结构上，进行布局布

线，达到在选定器件上实现设计的目的。依次执行 3 个主要任务：优化（opt_design）、布局（place_design）、布线（route_design）。

两种方法可以启动实现操作：一是在综合完成之后的对话框（图 3.27（c））上，选择“Run Implementation”并单击“OK”启动实现操作；二是在左侧导航栏，单击“IMPLEMENTATION”→“Run Implementation”，在弹出的对话框中，指定任务数并单击“OK”启动实现操作，如图 3.28 所示。

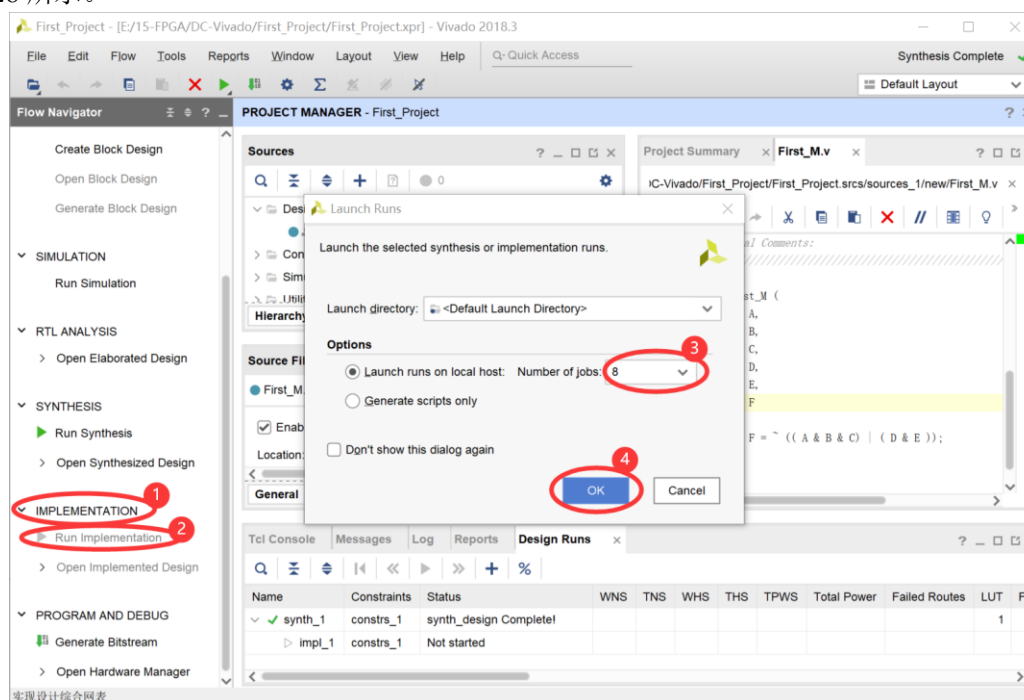


图3.28 执行实现操作

执行实现操作过程中，也会在窗口右上角显示当前执行的任务状态，完成后显示“Implementation Complete”。如果没有执行综合操作就执行实现操作，会弹出对话框提示“‘There is no netlist available. OK to launch synthesis first? Implementation will automatically start when synthesis completes.’”，点击“OK”就首先执行综合操作，再执行实现操作。

实现操作结束后，会弹出对话框，如图 3.29 所示，默认选择“Open Implemented Design”，单击“OK”可进入引脚约束的图形界面；如果引脚约束已经在实现前完成，这里可以选择“Generate Bitstream”，单击“OK”进入生成比特流阶段。实现之后可以查看电路图如图 3.30 所示，显然明确给出了使用的 FPGA 内具体逻辑单元。

此时，点击 Project Summary 页，可看到工程当前的状态，包括资源使用情况（Utilization），如图 3.31（a）所示。单击上面的蓝色字，均可以打开工程的设置（Settings）界面，如图 3.31（b）所示，可以修改相关信息，例如重新选择芯片型号、设置顶层模块等。工程设置界面也可以通过单击左侧工程管理器（PROJECT MANAGER）导航栏的“Settings”打开。

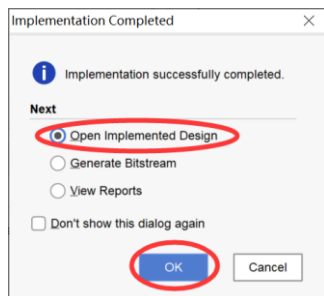


图3.29 实现完成后的对话框

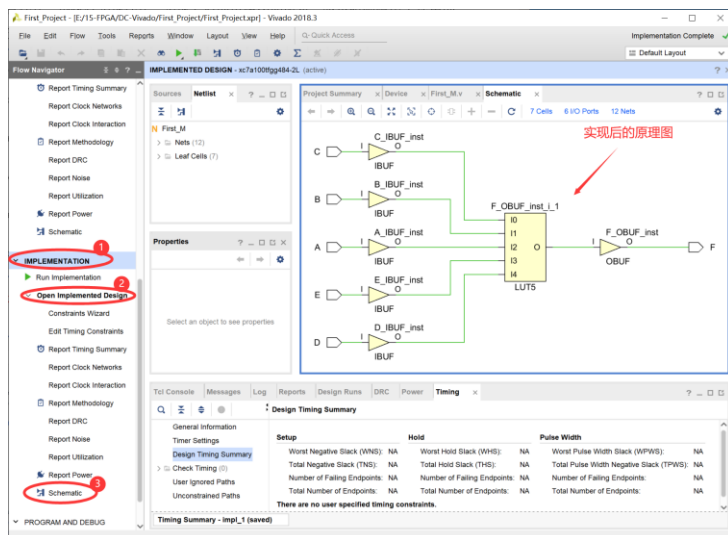
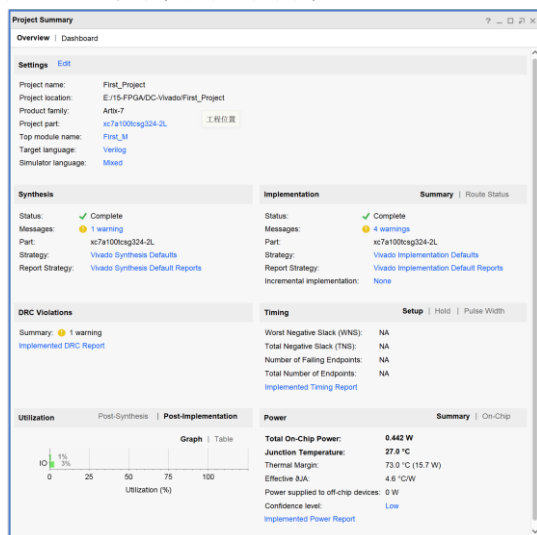
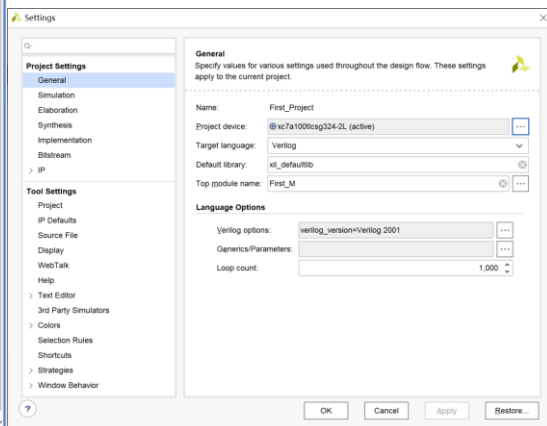


图3.30 实现后的原理图



(a) 工程状态总览



(b) 工程设置

图3.31 工程状态总览与工程设置

使用 Vivado 设计的电路，最终要变成比特流文件下载到 FPGA 芯片中去，但是要想正确产生比特流文件，必须首先要进行引脚约束。

3.2.8 引脚约束

1. 引脚约束概念

引脚约束又叫引脚配置，就是将工程顶层模块的输入输出端口与 FPGA 芯片的 I/O 引脚一一对应起来。FPGA 芯片的 I/O 引脚可以被配置成输入（input）、输出（output）或者双向（inout）三种类型，最终是哪种类型，取决于设计者如何进行引脚配置：如果将顶层模块的输入端口配置到 FPGA 的某个引脚上，该引脚就变为输入引脚；如果将顶层模块的输出端口配置到 FPGA 的某个引脚上，该引脚就变为输出引脚。

以 FPGA 芯片为主构造的开发电路板或者实验板卡上，通常会连接一些输入设备和输出设备到 FPGA 的 I/O 引脚上。因此，在进行引脚约束前，必须先要了解开发板或者实验板

上输入设备和输出设备具体连接到了哪些 FPGA 引脚上。常见的输入设备有开关、按键等，常见的输出设备有 LED 显示灯、数码管、触摸屏等。本教材介绍了 2 种实验板卡，具体的外设及其连接的 FPGA 芯片引脚介绍，见“第 4 章”及附录 A。

对于 First_M 模块，显然应当将 A~E 五个输入端口配置到输入开关所连接的 FPGA 引脚上，同时将输出端口 F 配置到 LED 灯连接的 FPGA 引脚上。

假设使用 HCS-A02 实验板卡完成 First_M 实验，且：选择下排最左边 5 个开关来输入 A~E 的值，用最左边的 1 个 LED 灯来显示 F 输出的值，那么这些设备的具体 FPGA 引脚是什么呢？可以通过观察板卡上设备上面或者下面的“引脚号”，或者查看第 4 章中的设备引脚列表来确定。最终得出结论：A~E 应配置到 FPGA 的 R8、T9、V1、V2、V4 引脚，F 应配置到 L3 引脚。如果使用其他的实验板卡，请阅读板卡手册，查找相应引脚。

2. 引脚约束方法

顶层模块的端口与 FPGA 引脚之间的配置关系以格式化的文本形式写在约束文件（Constraints File）中，即*.xdc 文件。有两种方式可以创建 XDC 文件：一种是新建（Create）约束文件并通过文本编辑方式输入，另一种是通过图形界面添加生成，二者是保持一致并相关联的。

（1）方法一：新建约束文件并手动输入约束代码

如图 3.32 所示，导航栏中单击“Add Sources”，在弹出的对话框内选择“Add or create constraints”，单击“Next”执行下一步。

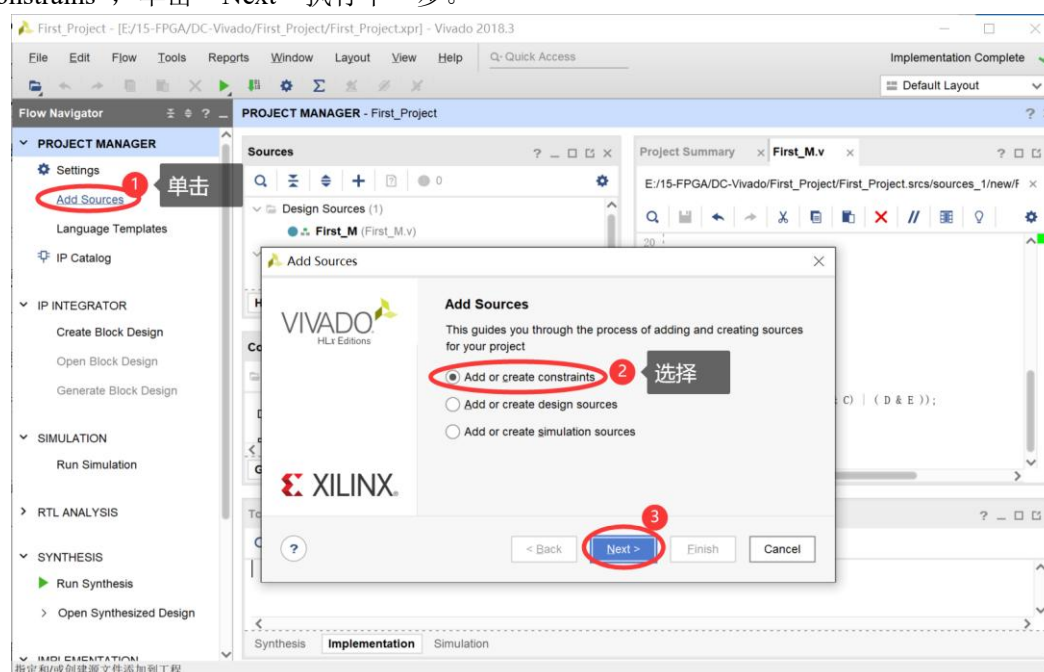


图3.32 新建约束文件

在弹出的对话框上（图 3.33）选择“Create File”，继续在新对话框内输入约束文件名“First_M”，最后单击“OK”后，对话框关闭，单击“Finish”就会创建一个名为“First_M.xdc”的空约束文件。

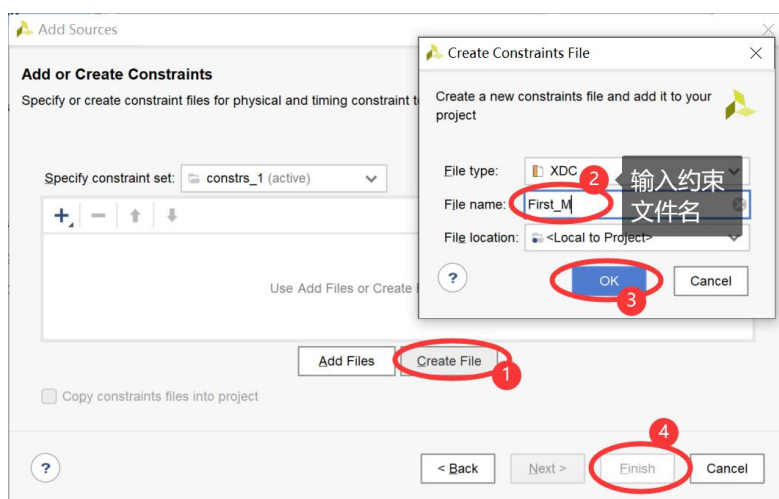


图3.33 输入约束文件名

这时在“Sources”子窗口的树形工程浏览器中，找到“Constrains”，在下面即可看到“First_M.xdc”，双击它打开程序编辑器，按照 XDC 文件的书写规范来定义引脚配置关系及引脚属性。

1) XDC 文件的格式与命令。下面是几种常用的设置端口属性的命令，其注释由“#”引导。

```
set_property PACKAGE_PIN R8 [get_ports A]      #设定模块的端口 A 配置到引脚 T3
set_property IOSTANDARD LVCMOS33 [get_ports A]  #设定端口 A 的标准电压为 3.3V
```

上述两句可以通过“-dict”合并为一句：

```
set_property -dict {IOSTANDARD LVCMOS33 PACKAGE_PIN R8 } [get_ports A]
#设定端口 A 的标准电压为 3.3V，并且配置到引脚 T3
```

如果在 Verilog HDL 程序中，检测了某个信号（例如 rst）的边沿（posedge rst 或者 negedge rst），那么该边沿信号应该配置到 FPGA 的专用时钟引脚（MRCC 和 SRCC）上，否则会报如下错误：

```
ERROR:[Place 30-574] Poor placement for routing between an IO pin and BUFG. If this sub optimal
condition is acceptable for this design,you may use the CLOCK_DEDICATED_ROUTE constraint in the .xdc
file to demote this message to a WARNING. However, the use of this override is highly discouraged. These
examples can be used directly in the .xdc file to override this clock rule.

set_property CLOCK_DEDICATED_ROUTE FALSE [get_nets rst_IBUF]
```

在 XDC 文件中，添加上述错误提示的最后一句话，即**关闭时钟引脚的指定布线**，就可以将“Error”降级为“Warning”，然后继续执行实现操作和生成比特流操作。其中的 rst_IBUF 是由系统自动生成的端口信号 rst 的缓冲信号。

2) HCS-A02 实验板卡的引脚约束规定。对于 HCS-A02 实验板卡，无论输入还是输出引脚的配置，必须指定电压为 3.3V，即：

```
set_property IOSTANDARD LVCMOS33 [get_ports F]
```

针对 HCS-A02 实验板卡的前述配置方案（A~E 配置到下排最左边 5 个开关，F 配置到最左边的 LED 灯），可以在程序编辑器中输入 First_M.xdc 文件的引脚约束代码，如下所示。

```
set_property -dict {IOSTANDARD LVCMOS33 PACKAGE_PIN R8 } [get_ports A]
set_property -dict {IOSTANDARD LVCMOS33 PACKAGE_PIN T9 } [get_ports B]
set_property -dict {IOSTANDARD LVCMOS33 PACKAGE_PIN V1 } [get_ports C]
set_property -dict {IOSTANDARD LVCMOS33 PACKAGE_PIN V2 } [get_ports D]
set_property -dict {IOSTANDARD LVCMOS33 PACKAGE_PIN V4 } [get_ports E]
set_property -dict {IOSTANDARD LVCMOS33 PACKAGE_PIN L3 } [get_ports F]
```

如果使用 NEXYS A7 实验板卡，假设仍旧采用上述配置方案，则 First_M.xdc 中的引脚约束代码则如下所示。

```
set_property -dict {IOSTANDARD LVCMOS18 PACKAGE_PIN V10} [get_ports A]
set_property -dict {IOSTANDARD LVCMOS18 PACKAGE_PIN U11} [get_ports B]
set_property -dict {IOSTANDARD LVCMOS18 PACKAGE_PIN U12} [get_ports C]
set_property -dict {IOSTANDARD LVCMOS18 PACKAGE_PIN H6} [get_ports D]
set_property -dict {IOSTANDARD LVCMOS18 PACKAGE_PIN T13} [get_ports E]
set_property -dict {IOSTANDARD LVCMOS18 PACKAGE_PIN V11} [get_ports A]
```

（2）方法二：I/O Planning 图形界面完成引脚配置

要进入 I/O Planning 图形界面，首先需要“**打开已实现的设计**”(Open Implemented Design)，打开的方法有三种：一是实现操作结束后，图 3.29 所示的对话框上选择“Open Implemented Design”，单击“OK”键；二是双击导航栏“IMPLEMENTATION→Open Implemented Design”；三是右键单击导航栏“IMPLEMENTATION”，弹出菜单上单击“Open Implemented Design→impl_1|xc7a100tfgg484-2L(active)”进入 I/O Planning 图形界面，这里还需要在下方的子窗口中，选择“I/O Ports”页。之后工程视图切换成图 3.34 所示的界面，这时我们注意到右上角自动切换成“I/O Planning”状态；如果不是该状态（例如在图 3.30 界面上为“Default Layout”），既可以通过下拉列表选择“I/O Planning”打开，也可以通过主菜单“Layout→I/O Planning”打开。

如图 3.34 所示，可以在下方子窗口的“I/O Ports”页展开“All Ports”，逐个输入或者选择各个端口对应的引脚，还可以选择该端口的电压、是否上拉/下拉等，全部配置好以后，单击保存按键，就会弹出一个对话框，其中输入 XDC 文件名“First_M”，最后单击“OK”键，就完成了“First_M.xdc”文件的创建。此时，选择 Sources 子窗口，单击工程树形图“Constraints”，可见已经多了一个约束文件“First_M.xdc”，双击它，在程序编辑器中可以看到：已经按照刚才的配置选择，自动填入了引脚约束代码，如图 3.35 所示。

因为添加了引脚约束，这时“综合”和“实现”均为过时的（**Out-of-date**），**需要重新执行综合和实现操作**。结束后，可进行下一步——生成比特流。

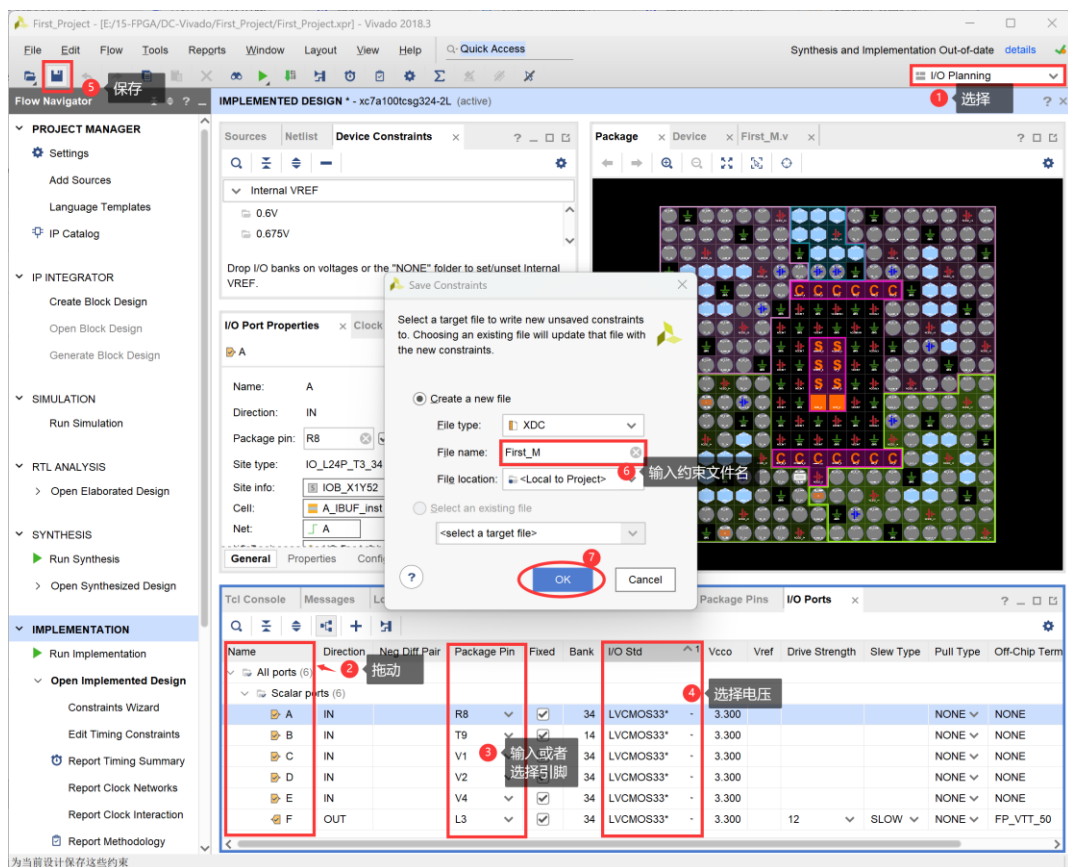


图3.34 I/O Planning 引脚配置图形界面

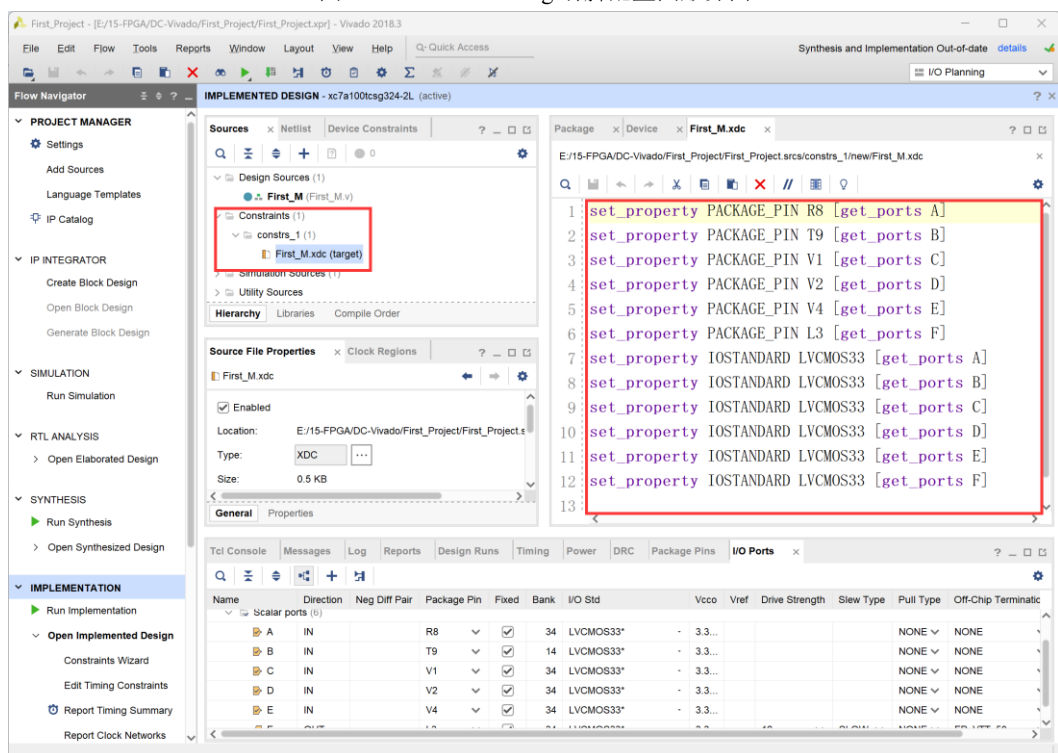


图3.35 I/O Planning 配置完成后的引脚约束代码

3.2.9 生成比特流文件与下载

硬件调试是 FPGA 开发最关键的一步，只有将 HDL 代码翻译成的硬件配置网表下载到 FPGA 芯片中，才能进行调试并最终实现相应的功能。首先必须生成能下载到硬件中的二进制比特流文件。

1. 生成比特流文件的方法

一种方法是在实现操作完成后，在图 3.29 所示的弹出对话框上，选择“Generate Bitstream”后单击“OK”，另一种方法是在左侧导航栏直接单击“PROGRAM AND DEBUG→Generate Bitstream”。这两种方法都会弹出一个对话框，如图 3.36 错误:未找到引用源。所示，输入任务数后，单击“OK”启动生成比特流的任务。如果此时综合和实现操作未完成，将弹出询问对话框，可依次自动执行综合操作、实现操作，最后再执行生成比特流任务。**如果工程设计未进行引脚配置，执行生成比特流任务将会最终失败（Failed）。**

生成比特流的任务执行完成后，将会产生比特流文件“*.bit”，也称为编程文件，本示例产生的是“First_M.bit”。

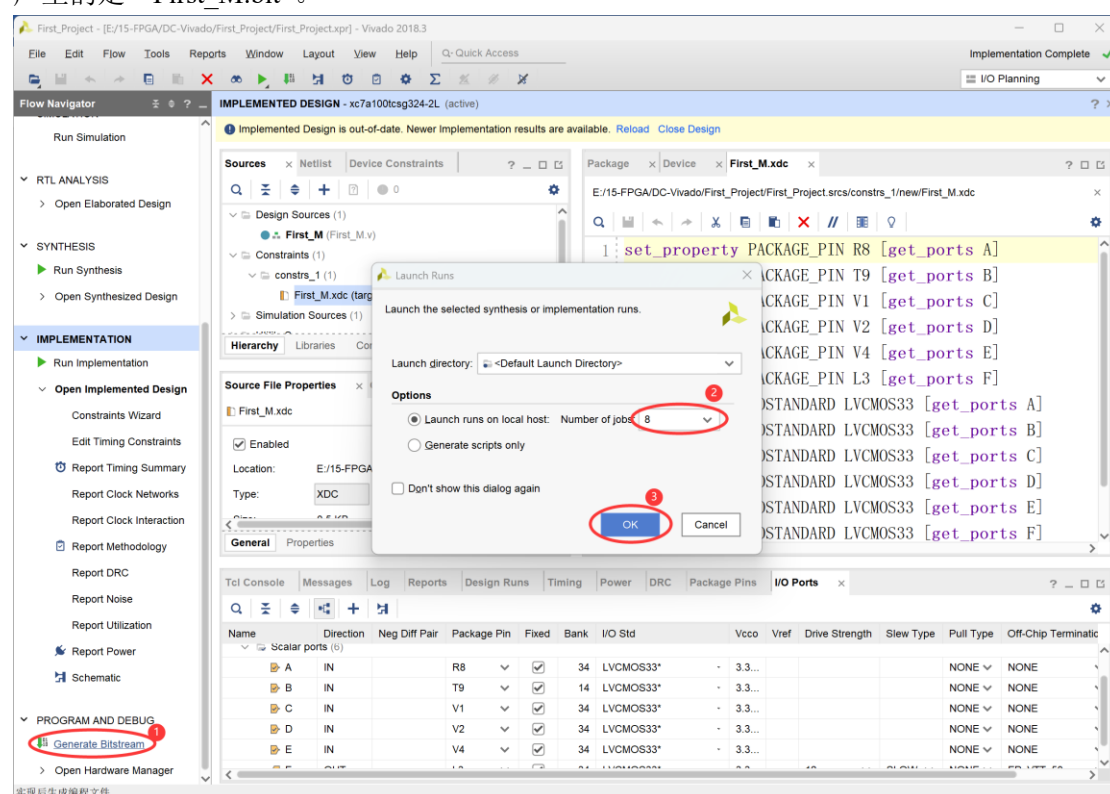


图3.36 启动生成比特流

2. FPGA 配置方式

生成的比特流文件需要下载到 FPGA 芯片中，才能让 FPGA 运行我们设计的电路。下载比特流文件到 FPGA 芯片这个过程又称为 FPGA 的**配置**（Configure），比特流文件称为**配置数据**。FPGA 处于工作状态时，配置数据存放在 FPGA 内部的 SRAM 查找表中，按照事先的设计电路运行。但 SRAM 是易失性存储器，因此，在 FPGA 上电后，需要外部电路

将配置数据重新装入（下载）FPGA 的 SRAM 中，这个过程称为配置状态。故一般的开发板均会为 FPGA 芯片配套设计一片非易失性存储器（常见的是 EEPROM、Flash）作为**配置存储器**，用于存放配置数据。

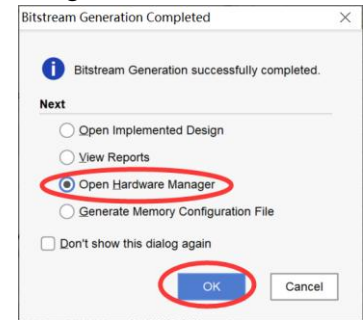
如何完成下载，取决于开发板和实验板的“外部电路”接口设计、FPGA 当前的配置方式。FPGA 下载方式通常有主动配置、被动配置和 JTAG 配置等几种，在物理上它们分别对应主串模式、从串/从并模式、边界扫描模式等比特流的烧写模式，由 FPGA 规定引脚上电平高低来选择（一般会接在在电路板上，通过短接器设置 0/1）。主动配置一般是用于产品设计完成后的正常运行态：FPGA 上电后，主动产生时序以读取配置存储器的配置数据（*.bit）并载入到 FPGA 的 SRAM，配置完成后，FPGA 进入工作状态。被动配置则由外部控制器（例如处理器）产生同步时序，并将自己提供的配置数据（通常为*.bin 文件）加载到 FPGA；被动配置常常用于远程配置，例如远程实验系统。JTAG 配置则用于开发调试阶段，通过 JTAG 接口直接把编程文件（*.bit）下载到 FPGA 上，断电后文件丢失。

3. 下载比特流文件*.bit 到 FPGA 芯片

本教材所提到的两种实验板卡上，均设计有 USB-JTAG 接口转换器，通过一根 USB 连接线连接电脑 USB 接口与实验板卡的 USB 编程接口；下载时，比特流文件通过板卡上的转换器，最后以 JTAG 接口协议下载到 FPGA 芯片上。

以 HCS-A02 板卡为例：下载前，先用 USB 数据线连接 HCS-A01 和计算机，注意方口连到板卡左上的 USB_PROG；然后上电，即板卡左上的电源开关拨至“开”位置；然后再执行下载操作。两种板卡均可以使用 Vivado 的硬件管理器（Hardware Manager）完成下载。

（1）打开硬件管理器。方法有两种：一是在生成比特流操作结束后，会弹出如图 3.37（a）所示的对话框，默认是“Open Implemented Design”选项，这里可以改选为“Open Hardware Manager”，单击“OK”打开；二是直接单击“PROGRAM AND DEBUG→Open Hardware Manager”（图 3.37（b））打开。打开之后的界面如图 3.38 所示。



（a）比特流文件生成完成



（b）“编程与调试”导航栏

图3.37 打开硬件管理器

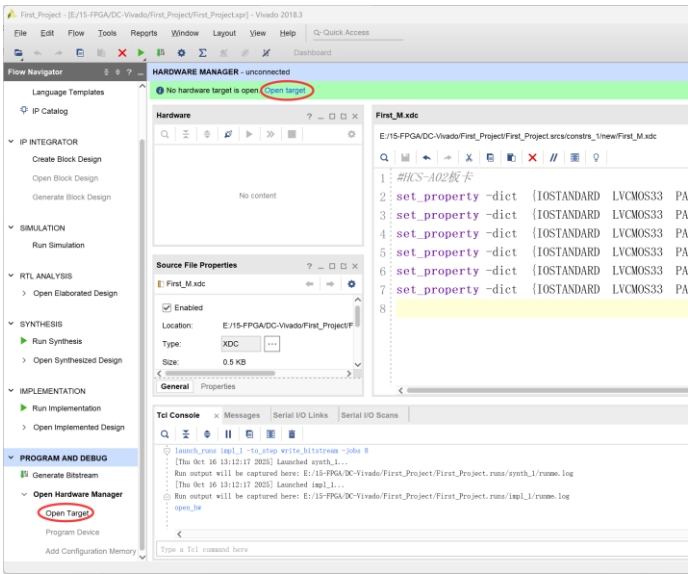


图3.38 硬件管理器界面

(2) **打开目标硬件。**用 USB 线连接好实验板卡并打开板卡电源开关后，可以单击“Open Target”，在弹出的菜单中选择“Auto Connect”，则会自动搜索目标硬件，搜索到后，会显示目标硬件的状态，如图 3.39 所示，当前器件处于“Not Programmed”状态。

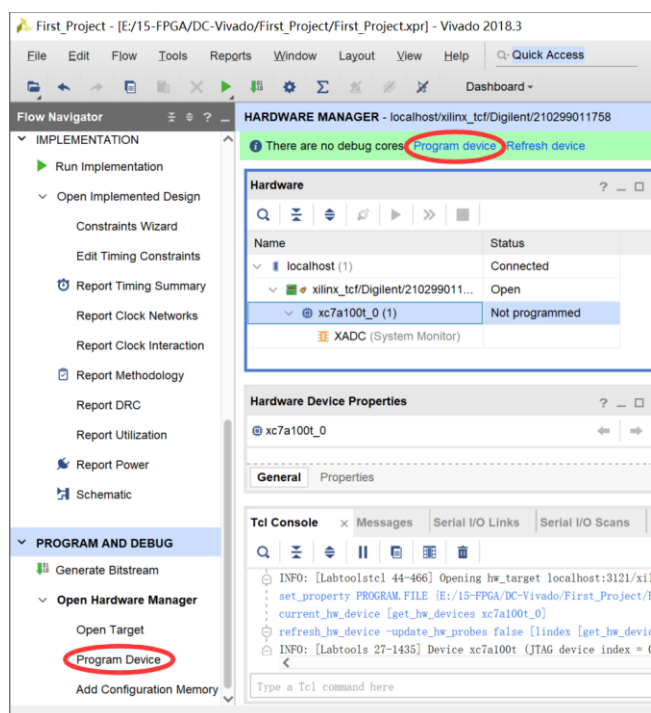


图3.39 目标硬件状态



图3.40 选择下载的*.bit 文件

(3) **编程 FPGA 器件。**继续单击“Program Device”，则会弹出对话框，如图 3.40 所示，单击“...”选择要下载的比特流文件 (*.bit)，再单击“Program”执行编程 FPGA 操作，完成下载。

如果经过板卡调试，需要修改程序设计或者引脚配置，可以单击左侧导航栏最上面的“PROJECT MANAGER”，切换窗口，显示工程源程序树形图。修改程序后，需要再次执行生成比特流的操作，这时依次会执行综合→实现→生成比特流三个阶段的任务。然后可以再次单击左侧导航栏最下面的“Program Device”实现编程下载。

4. 下载*.bin 文件到配置存储器

上一步骤，将比特流文件*.bit 下载到 FPGA 芯片后，只要不断电，即可开展板级的验证实验，检验程序是否正确、符合预期。断电后，如前所述，FPGA 中的*.bit 文件就不复存在。如果仅仅做实验，上述场景符合需求，但是如果作为一个产品应用，断电保持是基本需求。此时，应将程序下载到与 FPGA 配套的 EEPROM 或 Flash 中，则在断电后，会自动将程序加载到 FPGA 中运行。这里的程序是“*.bin”的二进制文件，要想生成*.bin 文件并下载到 EEPROM 中，则需要进行如下设置。

(1) **设置生成*.bin 文件的选项。**单击主菜单“Tools→Settings”，弹出对话框如图 3.41 所示。单击左侧的“Bitstream”条目，在右侧的选项中，勾选“-bin_file”，单击“OK”即可。然后再次执行“Generate Bitstream”生成比特流操作，此时，既生成*.bit 文件，也同

时生成“*.bin”文件。

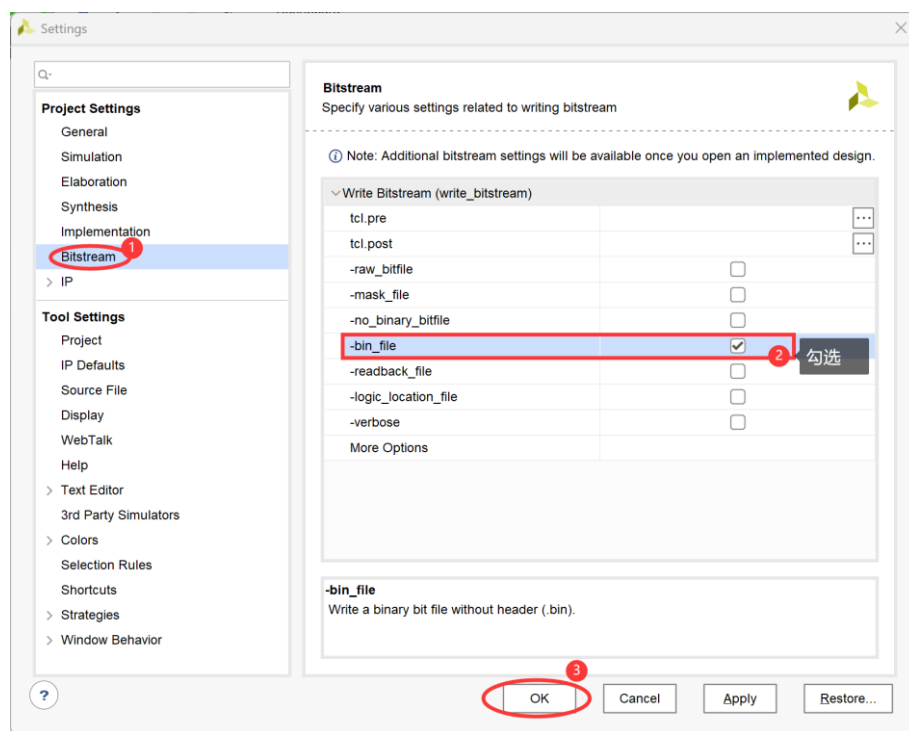
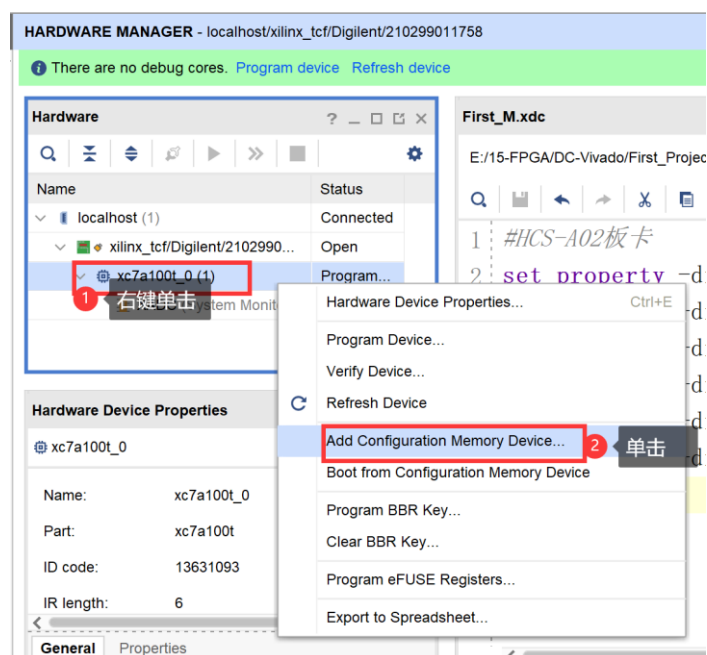
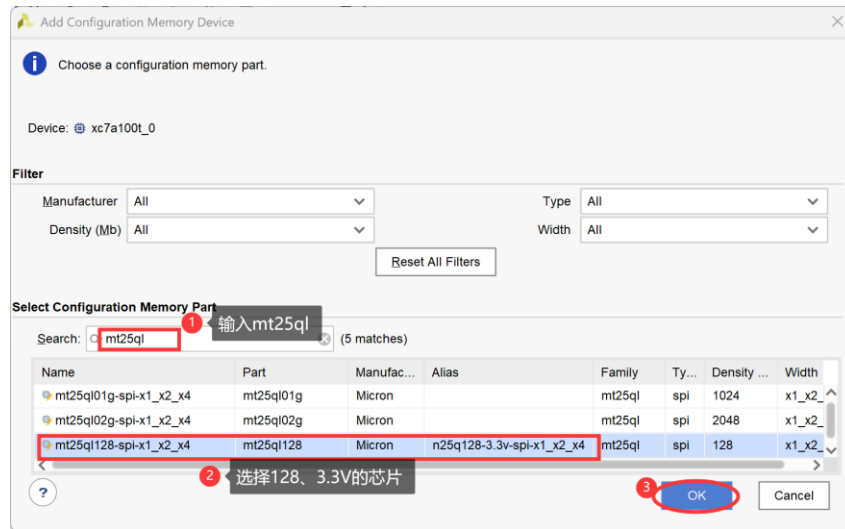


图3.41 设置生成*.bin 文件选项

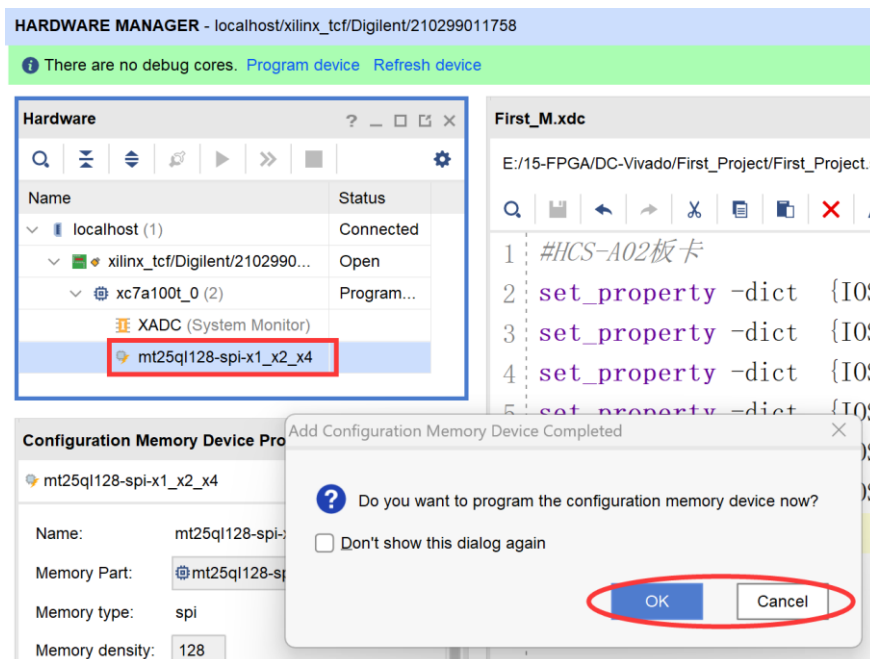
(2) 加载配置存储器芯片。如图 3.42 (a) 所示，在硬件管理器中，右键单击 FPGA 芯片 xc7a100t_0 (1)，在弹出菜单中，单击“Add Configuration Memory Device”菜单项，则会弹出图 3.42 (b) 所示的对话框；在 Search 文本框中输入“mt25ql”，则会有相符的芯片被筛选出来，选择“mt25ql128-spi-x1_x2_x4”这款芯片，单击 OK，则会弹出图 3.42 (c) 所示的对话框，可见已经加载好了 Flash 芯片，同时，询问是否现在就编程配置存储器芯片，可以依据需求来选择“OK”或者“Cancel”。



(a) 添加配置存储器



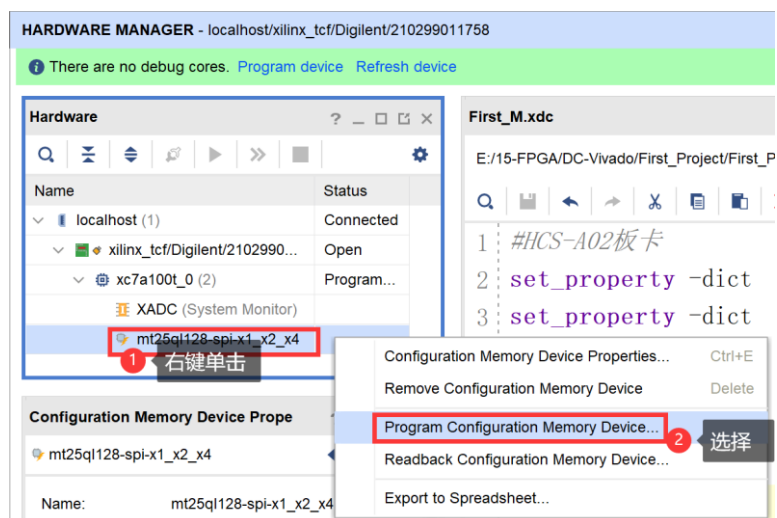
(b) 选择芯片型号



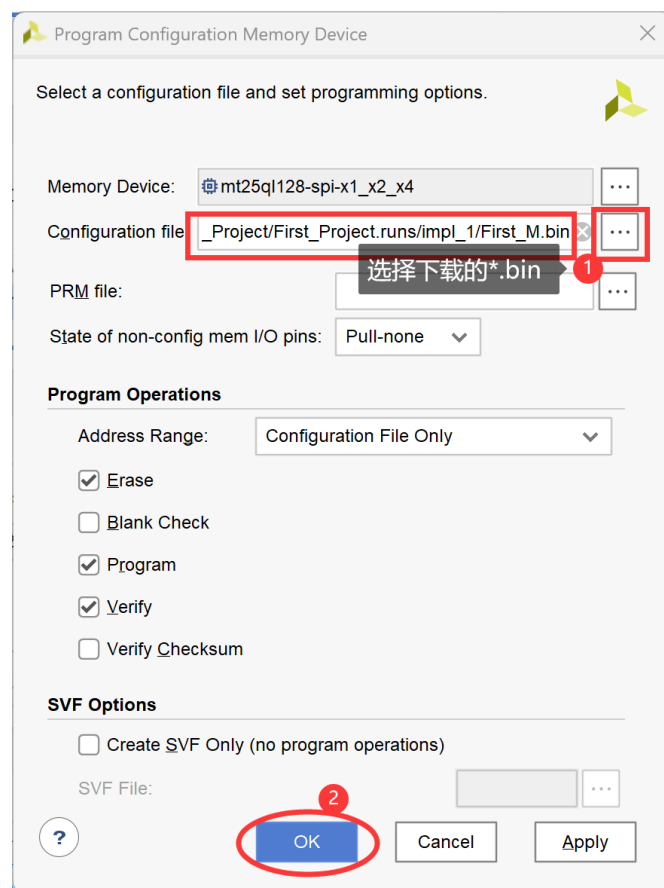
(c) 加载完成

图3.42 加载配置存储器芯片

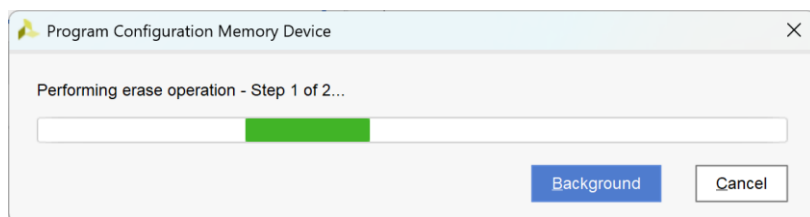
(3) 编程配置存储器。两种方法可以将*.bin 文件下载到配置存储器中。一种方法是在图 3.42 (c) 所示的加载配置存储器完成对话框中单击“OK”立即编程，另一种方法是在在硬件管理器中，如图 3.43 (a) 所示，右键单击配置芯片“mt25ql128-spi-x1_x2_x4”，在弹出菜单中，单击“Program Configuration Memory Device”菜单项，则会弹出图 3.43 (b) 所示的对话框；在其上选择前面已经生成的 First_M.bin 文件（一般会放在在工程目录 First_Project/First_Project.runs/impl_1/下），然后单击 OK，则开始执行编程配置存储器芯片的操作，如图 3.43 (c)；这个过程持续大约 2-3 分钟，先擦除、后编程，完成后显示提示框，即表明已经将*.bin 文件下载到配置 Flash 存储器中了。

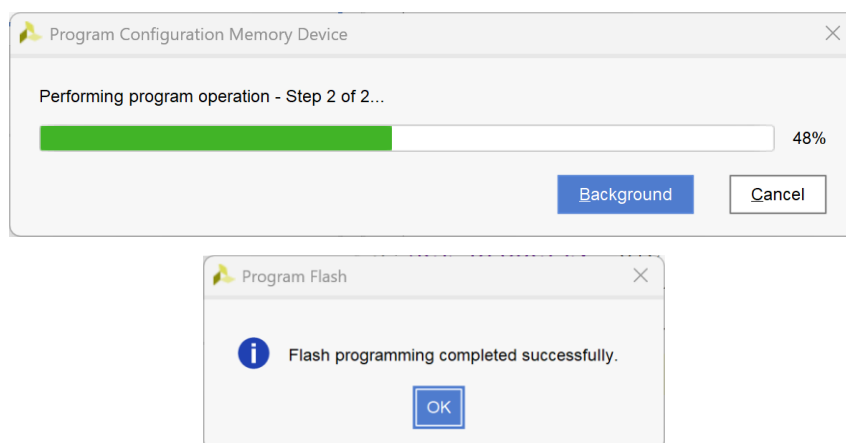


(a) 启动编程配置存储器任务



(b) 选择下载的*.bin 文件





(c) 执行先擦除和后编程两个操作

图3.43 编程配置存储器芯片

关闭 Vivado 软件的硬件管理器，关闭板卡电源，再上电，等待大约 10S 后，FPGA 芯片下方的配置完成指示灯点亮，表明已经将配置存储器中的程序加载到 FPGA 芯片中了。

NEXYS A7 实验板卡的下载方法和 HCS-A02 一样，并且还可以通过 Digilent Adept 软件来完成下载，请下载该软件，查找有关资料自行学习。

3.2.10 板卡实验

比特流文件下载到板卡后，FPGA 中的电路会按照我们的设计逻辑而运行。我们通过操作板卡上的输入设备，来观察板卡上相应的输出设备状态，是否正确。

对于前述的 First_M 示例，配置引脚时，A~E 配置到下排最左边的 5 位开关，而 F 配置到最左边的 LED 灯。则板级验证的过程就是：拨动下排最左边的 5 个开关，来观察最左边的 LED 灯是否符合 First_M 模块的输入、输出端口之间的逻辑关系： $F = \overline{ABC + DE}$ 。这里我们举两个例子验证其正确性。

当拨动下排最左边的 5 个开关（A~E）依次为 11000 时，即 ABCDE=11000，观察到最左边的 LED 灯亮了，即 F 输出为 1，符合 $F = \overline{ABC + DE}$ 的逻辑关系。

当拨动下排最左边的 5 个开关（A~E）依次为 00011 时，即 ABCDE=00011，观察到最左边的 LED 灯灭了，即 F 输出为 0，也符合 $F = \overline{ABC + DE}$ 的逻辑关系。

这两个例子对于 First_M 的逻辑而言是正确的，但是如果完全证明其正确性，还需要输入其他的 30 个输入组合，此处不赘述，请读者自行验证。

3.2.11 Vivado 工程文件目录与子目录

前述创建的工程文件名是 First_Porject，则在 First_Porject 目录下，有如图 3.44 所示的子目录和文件。

名称	修改日期	类型
First_Project.cache	2023/9/28 10:30	文件夹
First_Project.hw	2023/9/28 10:30	文件夹
First_Project.ioplanning	2023/9/28 10:30	文件夹
First_Project.ip_user_files	2023/9/28 10:30	文件夹
First_Project.runs	2023/9/28 10:30	文件夹
First_Project.sim	2023/9/28 10:30	文件夹
First_Project.srscs	2023/9/28 10:30	文件夹
First_Project.xpr	2023/9/28 10:33	Vivado Project File

图3.44 First_Porject 工程目录下的内容

其中，First_Porject.cache 存放了 Vivado 软件的运行缓存；First_Porject.hw 存放工程所有的波形文件；First_Porject.ioplanning 存放图形界面配置引脚的文件；First_Porject.ip_user_files 存放工程设计中使用的 IP 核实例的文件；First_Porject.runs 存放综合、实现与生成比特流的过程文件与结果文件；First_Porject.sim 存放仿真过程文件和仿真结果；First_Porject.srscs 存放所有编写的设计源程序、仿真文件与约束文件。First_Porject.xpr 是工程启动文件，双击它可以直接启动 Vivado 软件，并打开该工程，进入管理界面。下面重点介绍其中的两个子目录的相关内容。

(1) **First_Project.srscs**: 存放工程文件的三类源程序代码，分别存放在如下三个子目录内。

- **sources_1→new**: 存放所有的*.v 设计源程序文件，包括顶层模块、子模块的.v 文件，本次示例是 First_M.v 文件。
- **sim_1→new**: 存放所有的*.v 仿真源程序文件，本次示例是 First_M_Test.v 文件。
- **constrs_1→new**: 存放所有的*.xdc 引脚配置源程序文件，本次示例是 First_M.xdc 文件。

(2) **First_Project.runs**: 存放综合和实现操作的过程文件、比特流文件等，有如下三个子目录。

- **synth_1**: 含综合操作的过程文件和产生的综合网表文件 (*.dcp 文件)，本示例是 First_M.dcp 文件等。
- **impl_1**: 一是包含实现操作的过程文件和产生的网表文件 (*.dcp 文件)，本示例是 First_M_opt.dcp、First_M_placed.dcp、First_M_routed.dcp 文件等；二是包含生成比特流的过程文件和结果文件 (*.bit)，本示例是 First_M.bit 文件等；如果勾选了允许同时生成二进制文件，则此处还包含*.bin 文件，本示例是 First_M.bin 文件。
- **.jobs**: 存放每次执行任务的一个配置文件 (*.xml)，记录执行的参数等。

可见，Vivado 综合和实现操作都产生同一种格式文件，即“*.dcp”文件，包含了对应阶段处理的信息；每个阶段的 dcp 文件都是下一阶段的输入文件，dcp 文件可以通过菜单“File→Checkpoint→Open”打开。

事实上，设计者最常使用的是.srscs 子目录，含设计者编写的所有源代码（设计*.v、仿真*.v 和引脚配置*.xdc）；另外，下载比特流文件时，之前生成的比特流文件*.bit 和二进制

文件*.bin 存放在.runs→impl_1 子目录下。

习题 3

- 3.1 请下载安装 Vivado 软件，并按照本章 3.2 节的流程，完成工程创建，并对 First_M 完成全仿真。
- 3.2 使用实验板卡，完成 First_M 的板级验证实验。
- 3.3 动手实践，完成“3.2.5 行为仿真”中的时序逻辑电路的仿真案例，观察你的波形是否与图 3.26 一致？如果不是，分析原因。
- 3.4 模仿“3.2.5 行为仿真”中的时序逻辑电路的仿真案例，尝试完成【例 2.16】中的带同步清零端的正边沿触发器的 UP_DFF_RD_syn 模块的仿真，注意测试样例的完备性。
- 3.5 请重新创建一个工程，编写程序实现五输入表决器的控制逻辑，并完成仿真验证、板级验证。（注：五输入表决器的逻辑是：当输入的 5 个变量有 3 个及以上为“1”时，输出就为“1”；如果输入的 5 个变量为“1”的个数不足 3 个时，则输出为“0”。简言之：少数服从多数。）
- 3.6 力所能及，动手在 Vivado 中完成第 2 章的【思考与实践】环节，得出自己的结论。